

分类号: _____

密级: _____

UDC: _____

编号: _____

工学硕士学位论文

基于改进 MC 的海洋要素三维可视化研究

硕士研究生：朱智强

指导教师：高峰 副教授

学科、专业：控制科学与工程

论文主审人：

哈尔滨工程大学

2024 年 5 月

分类号: _____

密级: _____

UDC: _____

编号: _____

工学硕士学位论文

基于改进 MC 的海洋要素三维可视化研究

硕士研究生：朱智强

指导教师：高峰 副教授

学位级别：工学硕士

学科、专业：控制科学与工程

所在单位：青岛创新发展基地

论文提交日期：2024 年 4 月

论文答辩日期：2024 年 5 月

学位授予单位：哈尔滨工程大学

Classified Index:

U.D.C:

A Dissertation for the Degree of M. Eng

Research on 3D Visualization of Marine Elements
Based on Improved MC

Candidate: Zhu Zhiqiang

Supervisor: Associate professor Gao Feng

Professional category: Control Science and Engineering

Date of Submission: Apr, 2024

Date of Oral Examination: May, 2024

University: Harbin Engineering University

摘 要

可视化技术在海洋环境研究领域具有重要的作用,不仅能扩展人类对海洋的认知和理解,还提供了一种直观、高效的分析工具,可以更直观的获取海洋中信息的变化。然而,海洋环境是个极其复杂的环境,海水中存在多种海洋要素,具有变化复杂多样等诸多特点;同时,随着观测技术的发展和进步,能够获得的海洋要素数据也呈爆炸性态势。面对庞大的计算量与高精度图像的绘制要求,如何快速且准确的绘制出海洋要素三维图像成为了一个关键问题。

目前体视化方法有很多,面绘制中的MC(Marching Cubes,行军立方体)算法具有精度高、计算效率相对较快等特点,非常适合海洋要素三维可视化处理。因此,本文基于MC算法进行海洋要素的三维可视化研究。首先对MC算法的基本原理及研究现状进行了详细分析;在此基础上,依据海洋要素的特点,针对传统MC算法存在拓扑二义性无法确定,效率存在优化空间等问题进行了改进;最后基于改进的MC算法设计并实现了海洋三维可视化系统。具体研究内容如下:

(1) 针对传统MC算法中拓扑二义性无法保证的缺点,提出一种细分拓扑结构的方法进行处理。对拓扑结构错误等问题进行了深入的分析,针对容易产生二义性的一些拓扑情况进行准确细分。通过强制规定二义性面正连接的方式,确保与相邻体素的连接不出现错误,保证拓扑正确性。

(2) 采用优化八叉树数据结构对MC算法进行改进。由于传统MC算法需要遍历所有体素的特点,八叉树数据结构对算法进行剖分递归运算,能够规避大量空白体素的计算,通过针对海洋要素特点进行合理设置递归终止条件,使运算效率得到了提升。

(3) 在提升MC算法效率之外,充分考虑计算机硬件条件,提出一种多线程并行运算方法。通过将拓扑构型分为四个大类,使用不同的线程进行针对处理,使得算法效率得到了进一步提升。

(4) 基于改进 MC 算法设计并实现了一套海洋要素三维可视化系统。系统分为数据载入,等值面提取,图像绘制三大模块。通过前后端分离的形式完成,前端基于 Cesium 框架设计而成。后端基于 SpringBoot 框架搭建,并进行封装打包,拥有跨平台的特性。

关键词: 海洋要素三维可视化; 拓扑构型; 改进MC算法; 多线程; 八叉树

ABSTRACT

Visualization technology plays an important role in the field of marine environmental research, not only expanding human cognition and understanding of the ocean, but also providing an intuitive and efficient analysis tool that can more intuitively obtain changes in information in the ocean. However, the marine environment is an extremely complex one, with various marine elements present in seawater, which have many characteristics such as complex and diverse changes; Meanwhile, with the development and progress of observation technology, the available ocean element data is also showing an explosive trend. Faced with the huge computational workload and the demand for high-precision image rendering, how to quickly and accurately draw three-dimensional images of ocean features has become a key issue.

At present, there are many methods for volume visualization, and the MC (Marching Cubes) algorithm in surface rendering has the characteristics of high accuracy and relatively fast computational efficiency, making it very suitable for 3D visualization processing of ocean features. Therefore, this article is based on the MC algorithm to study the three-dimensional visualization of marine elements. Firstly, a detailed analysis was conducted on the basic principles and research status of the MC algorithm; On this basis, based on the characteristics of marine elements, improvements have been made to address the problems of traditional MC algorithms such as uncertain topological ambiguity and optimization space in efficiency; Finally, an ocean 3D visualization system was designed and implemented based on the improved MC algorithm. The specific research content is as follows:

(1) A method of subdividing topological structures is proposed to address the shortcomings of traditional MC algorithms that cannot guarantee topological ambiguity. An in-depth analysis was conducted on issues such as topology errors, and accurate segmentation was carried out for some topology situations that are prone to ambiguity. By mandating the positive connection of ambiguous faces, it ensures that there are no errors in the connection with adjacent voxels and ensures topological correctness.

(2) Using optimized octree data structure to improve the MC algorithm. Due to the traditional MC algorithm's need to traverse all voxels, the octree data structure performs recursive partitioning operations on the algorithm, which can avoid the calculation of a large number of blank voxels. By setting reasonable recursive termination conditions based on the characteristics of marine elements, the efficiency of the operation is improved.

(3) In addition to improving the efficiency of the MC algorithm, a multi-threaded parallel

computing method is proposed by fully considering computer hardware conditions. By dividing the topology configuration into four categories and using different threads for targeted processing, the efficiency of the algorithm has been further improved.

(4) A three-dimensional visualization system for ocean features was designed and implemented based on an improved MC algorithm. The system is divided into three modules: data loading, contour extraction, and image drawing. Completed through front-end and back-end separation, the front-end is designed based on the Cesium framework. The backend is built based on the SpringBoot framework and packaged, with cross platform features.

Key words: 3D visualization of marine elements; Optimize Topological configuration; MC algorithm; Multithreading; Octree

目 录

第 1 章 绪论.....	1
1.1 课题研究背景及意义.....	1
1.2 课题的国内外研究现状.....	2
1.2.1 海洋数据体视化方法研究现状.....	2
1.2.2 MC 算法研究现状	3
1.3 论文的主要研究内容及章节安排	4
第 2 章 体视化相关理论与方法	7
2.1 体视化介绍.....	7
2.2 三维可视化方法分类.....	8
2.2.1 直接体绘制.....	9
2.2.2 面绘制.....	11
2.3 海洋体数据场.....	12
2.3.1 体素的介绍.....	12
2.3.2 体数据场.....	13
2.4 传统 MC 算法	14
2.4.1 传统 MC 算法的原理	14
2.4.2 传统 MC 算法存在的问题	19
2.5 MC 算法的优化方向	20
2.6 本章小结.....	20
第 3 章 MC 算法的改进	21
3.1 拓扑二义性改进.....	21
3.1.1 空间拓扑关系.....	21
3.1.2 拓扑二义性的由来.....	22
3.2 MC 算法的拓扑正确性改进	23
3.2.1 改进方向分析.....	23
3.2.2 拓扑正确性验证.....	25
3.3 基于八叉树数据结构改进 MC 算法	26
3.4 八叉树数据结构的程序实现	28
3.4.1 节点面片的判定.....	29
3.4.2 递归终止判定.....	30
3.5 仿真验证.....	31
3.5.1 改进 MC 算法的精确性验证	32
3.5.2 改进 MC 算法的速度验证	33
3.6 本章小结.....	34

第 4 章 基于并行运算的效率优化	37
4.1 多线程的基本概念	37
4.1.1 进程与线程	37
4.1.2 同步与异步	39
4.2 多线程程序设计	41
4.2.1 优化流程分析	41
4.2.2 拓扑构型的分类处理	43
4.3 多线程程序实现	46
4.3.1 数据共享区设计	46
4.3.2 多线程并行速度验证与分析	47
4.4 跨平台验证运行效率	48
4.5 本章小结	48
第 5 章 基于改进 MC 算法的海洋可视化系统设计与实现	49
5.1 海洋可视化平台研究现状	49
5.2 系统整体框架设计	50
5.2.1 系统需求分析	50
5.2.2 系统框架设计	50
5.3 基于改进 MC 算法的海洋可视化系统	51
5.3.1 海洋数据读取	51
5.3.2 基于 Spring Boot 的后端系统搭建	52
5.3.3 基于 Cesium 的前端页面搭建	53
5.3.4 可视化系统测试	55
5.4 本章小结	56
结论	57
参考文献	59

第1章 绪论

1.1 课题研究背景及意义

海洋环境是个极其复杂的环境，海水中存在多种海洋要素，如海温、盐度、密度等变化复杂多样^[1]。随着人们对海洋观测的深入，能够获得的海洋要素数据越来越多，研究海洋生态系统中的环境要素，是认识其空间分布特征及作用机理的关键^[2]。传统的观测手段往往受限于时间、空间和成本等因素，无法细致的展现海洋各项数据的内容。视觉是人们获得外部世界的最重要方式，80%以上的外来信息都是经过视觉加工的^[3]，人眼对图像的处理具有速度快、容量大、可同时处理等特性，因此可视化研究变得逐渐热门。

在传统可视化的研究中，可视化技术可以按照其具体的用途划分为：科技计算可视化与资讯可视化^[4]。其中，科技计算可视化侧重于 2D、3D 领域的图表示，而资讯可视化侧重于抽象的数据库与数据仓库的视觉表达。通过可视化，研究人员可以更直观地观察海洋环境的变化趋势、发现隐藏的规律^[5]，并为海洋资源管理、环境保护和灾害预防等方面提供决策支持。因此，可视化在海洋环境研究中具有重要的作用。它不仅扩展了人们对海洋环境的认知和理解，还提供了一种直观、高效的分析工具，推动了海洋科学的发展和应用。

在海洋可视化领域中有一种等值面的绘制，能够帮人们快速的看到海洋各种要素的数据变化形式，研究海洋内部信息的各种规律。随着技术的更新，人们获取到的海洋各种要素数据呈现范围广，分辨率高的特点，导致数据量急剧增加。而等值面的绘制中 MC 算法最为经典^[6]，但传统 MC 算法存在拓扑二义性无法确定，效率存在优化空间等诸多问题^[7-10]。虽然一直以来有诸多学者对 MC 算法进行创新改进，但缺乏针对海洋要素的优化。

本课题对海洋数据的三维可视化技术进行研究，并结合海洋水文要素数据的特点^[11]提出基于改进 MC 算法的海洋等值面三维可视化方法；针对该方法在绘制拓扑二义性无法保证的不足，对其进行改进，仿真结果表明改进方法有效保证拓扑二义性的正确性，在面对复杂鞍面等情况都能正确绘制；针对该方法在绘制速率方面的不足，对其进行改进，仿真结果表明改进方法有效提升传统 MC 算法的运算效率，与此同时，通过针对计算机硬件领域的发展调整算法，进一步提升 MC 算法的性能；最后在此基础上，完成海

洋三维可视化系统的设计与开发,为用户能够方便的载入数据,对需要感知的海洋数据进行三维可视化绘制提供帮助。

1.2 课题的国内外研究现状

1.2.1 海洋数据体视化方法研究现状

海洋环境研究者通常需要分析海洋环境的规律和变化趋势,通过动画、交互式可视化和虚拟现实等技术,将数据以动态的方式展示,使研究者能够更好地理解海洋环境的时空变化规律^[12]。其次,数据可视化方法从二维逐渐拓展到多维^[13]。在海洋环境研究中,海洋数据通常具有多个维度,例如空间、时间和不同环境要素之间的关系。为更好地理解这些多维数据,研究者开发了各种方法和技术,例如多维图表、平行坐标和热力图等,以更全面地呈现和分析海洋环境数据。此外,数据可视化方法也从简单的基本属性信息表达逐渐发展到分析成果结论的呈现。在海洋环境研究中,研究者不仅需要对数据进行可视化展示,还需要从中提取规律和结论。为此,他们开发了各种分析工具和算法,以便更好地理解 and 解释海洋环境数据的含义。通过可视化方法,海洋环境研究者能够更加直观地观察和分析海洋环境的变化趋势,并从中提取出有价值的信息。这种可视化分析不仅有助于揭示海洋环境的规律和机制,还为海洋资源管理和环境保护等方面提供重要的决策支持。随着数据可视化技术的不断发展和创新,人们可以期待在海洋环境研究中获得更深入和全面的认识。

三维数据建模是将数据组织、处理方法和规则的限制等方面的综合。自 1960 年中期到现在,关于 3D 建模的研究取得丰硕的成果。其中最具典型意义的有:1987 年,Carlson Eric^[14,15]从地质观点出发,建立一个立体的地下空间结构模型,并建立一个简单复合体(Simplicial Complex)。1992 年 Molenaar Martien^[16]在原有的 2D 空间中引入由节点、弧、边、面等 4 要素构成的 3DFDS。Pilouk M, 陈琳^[17-19]等人提出一种以四面体为基础的三维数据模型(TEN)。W.W.等^[20]将三维网格、不规则块体、断面及体等三维空间信息相融合,建立适用于分层构造的三角柱体(TP)模型。Zlatanova^[21]建立一种简单的空间数据模型(SSM)。Coors V^[22]在此基础上,建立一种以曲面为代表的立体曲面表示的城市数据模型(UDM)。在国内,3D 数据建模技术的发展相对滞后,具有典型意义的有:陈筱柳等^[23]提出一种新的 3D 向量数据模式(TEN);史文中^[24]以正方形网格和八叉树为基础,对 TIN-Octree 进行研究;彭福玲等^[25]对基于对象的栅格矢量融合的 3D 数据建模进行研究。龚健雅、程朋根^[26,27]以矿区地质为基础,并与勘查项目相联系,

进行矢网融合的面向对象混合数据建模方法研究；张煌、吴立新、齐安文、程朋根等^[28,29]对基于三角形圆柱，类似（或推广）三角形圆柱的 3D 数据模型进行研究。侯恩科^[30]等人对用于地球模拟的 3D 体型元素的拓扑数据模式进行研究；史文中^[31]建议一种面向目标的 3D 资料模式（O3D）来处理复杂的 3D 物体；俞肇元拟探讨高维空间事件的适应性表示和融合模型，构造具有无交集的多维统一空间标（BRNO-ST），并将其应用于实际应用中。吴立新^[31]等人以球面退化八叉树网格（SDOG）为基础，构建并开发能够对地球内部、地表和地表的系统建模、表达和分析的 SDOGESSG 模式。

1.2.2 MC 算法研究现状

MC 算法是一种利用三角形块来表示其最基础的图形单元的方法，MC 算法本身原理简单，易于实现。近年来，国内外学者对其进行大量的研究，并取得较好的效果。但是，目前 MC 方法仍局限于对确定拓扑正确^[32-39]，提高渲染精度^[40-42]，减小存储空间，提高运算速率^[43-49]。至今，国际知名期刊及学术研讨会上仍有许多关于 MC 算法的改良方案。由 Van Gelder et al.^[35]讨论由等价曲面提取引起的拓扑结构问题，Brodie et al.^[41]总结多种计算方法和改进的方法。Durst 通过对基础体素物态模型的研究，认为在一个立方体元体的某一平面中，若该平面中的某一平面中的点与该平面之外的各点，则存在 2 种连接方法；由于两个相邻的两个立方在一个平面内的连通方式有差异，因此，在两个或更多的连通方式中选取合适的一个是一个难点。Nielson^[38]建议采用一种新的双曲型渐进线方法，以消除表面的模糊性，对三次曲线内插值函数在立方体内的特殊形式作细致的研究，给出初步的解答。Natarajan 引进“面”与“体鞍”两个新的定义，并将其应用于体二像性研究中。

Van Gelder et al.^[35]研究表明：为确保近似多边形的边界仅限于 2 个多边形共存库的边仅存在于一个近似多边形中；Cignoni 在 Natarajan 等人关于面鞍点与体鞍点的研究工作的基础上，将算法中的基元态划分成扩充的查表法，并针对不同情形提出相应的三角化方法。Lopes et al.^[44,45]采用肩部点改进等值曲面的计算方法，该方法将等值曲面和立方相交线所构成的边缘多边形的每一条边都位于该立方的平面内，并通过添加肩部点对其进行延拓，进一步提升该曲面在曲面上的表达精度。Lopes 方法是在立方体内添加一些顶点，从而使近似的等高线在立方体内的表达更加精确。在此基础上，Cignoni 设计一种改进的网格提取方法。薛强 et al.^[38]针对体积数据，给出一种基于 Marching Boxes（MB）的体积数据域的多分辨方法，此方法将整体的体积数据域看作一个盒子，并将盒子分为 Box1 和 Box2，通过 MC 方法提取它们之间的等值平面；Pascucci et al.介绍一

种以交互为基础的多解析度算法。现有的多分辨方法构建等值面耗时较长，无法确保连续改变的等值面间的互动展示，也无法对所产生的各层面的几何形态进行有效的控制。

综上所述，传统 MC 算法对于等值面的绘制虽然简便可行，但是也有一些不足，具体为以下几点：

(1) 无法确保拓扑二义性：很多时候拓扑结构的三角面片的连接会出现错误，具体展现在图像上是出现“空洞”等直观错误，或是一些细节方面的不同，无法完美绘制出原本的图像。

(2) 计算效率不足：在所有三维体绘制方法中，MC 算法的速率相对较快，但算法本身还有诸多可以优化提升的地方，例如需要遍历大量空白体素等。

(3) 没有对计算机硬件发展提出相应的改进：MC 算法是上个世纪末诞生的经典可视化算法，由于近几十年来计算机领域的高速发展，计算机计算效率以及计算方式等发生了许多改变，这对 MC 算法来说是可以改进的方向之一。

(4) 对海洋要素可视化的针对性改进：在 MC 算法的研究发展中，有诸多学者在各个不同的应用领域算法进行针对性的改进，但大部分为医疗应用、地质勘探等领域。针对海洋环境与海洋要素自身的特点进行改进成为改进的方向之一。

1.3 论文的主要研究内容及章节安排

MC 算法是简单实用的三维可视化方法，且后来有很多学者对其进行相应的改进，但大多都是 MC 算法的改进都是围绕医学领域的研究，在针对海洋三维可视化领域缺乏相应的研究与分析。本文旨在针对海洋三维可视化方面，对传统 MC 算法改进，解决其速度慢、正确性差等诸多问题，并以此设计海洋三维可视化系统。本文的第一部分是对比各种海洋可视化方法，阐述传统 MC 算法的基本原理，并分析算法本身的缺点与改进方向；第二部分使用细分拓扑结构和优化八叉树数据结构对传统 MC 算法进行改进；第三部分使用多线程并行处理，进一步提升算法效率，使用插值手段调整图像分辨率；第四部分基于优化后的 MC 算法设计海洋三维可视化系统。

论文各个章节安排如下：

第 1 章：绪论。首先介绍海洋三维可视化的研究背景和研究意义，之后分析海洋三维可视化相关技术的研究现状，最后提出本文的主要研究内容和章节安排。

第 2 章：体视化相关理论与方法。本章先是分析各种三维可视化方法的具体优缺点，之后介绍海洋体数据场的基本特点，最后阐述传统 MC 算法的基本原理与算法的各方面

缺点，并指出优化方向。

第3章：MC算法的改进。本章主要针对传统MC算法的拓扑二义性无法保证的缺点，采用细分拓扑结构的方式进行改进；针对算法效率的不足，采用八叉树数据结构进行改进，避免大量空白体素的遍历，提升算法效率，压缩存储空间。

第4章：基于并行运算的效率优化。本章主要使用多线程并行计算的方法对算法进行效率优化，通过分析海洋要素等值面的特点，将拓扑构型分为四个大类，使用不同的线程进行针对处理，使得算法效率得到了进一步提升。

第5章：基于改进MC算法的海洋可视化系统设计与实现。本章先是分析现有的海洋可视化平台，然后对系统整体框架进行设计，最后基于本文改进的MC算法，针对海洋数据的特点设计并开发了海洋要素三维可视化系统。

本文的结构安排如图1.1所示。

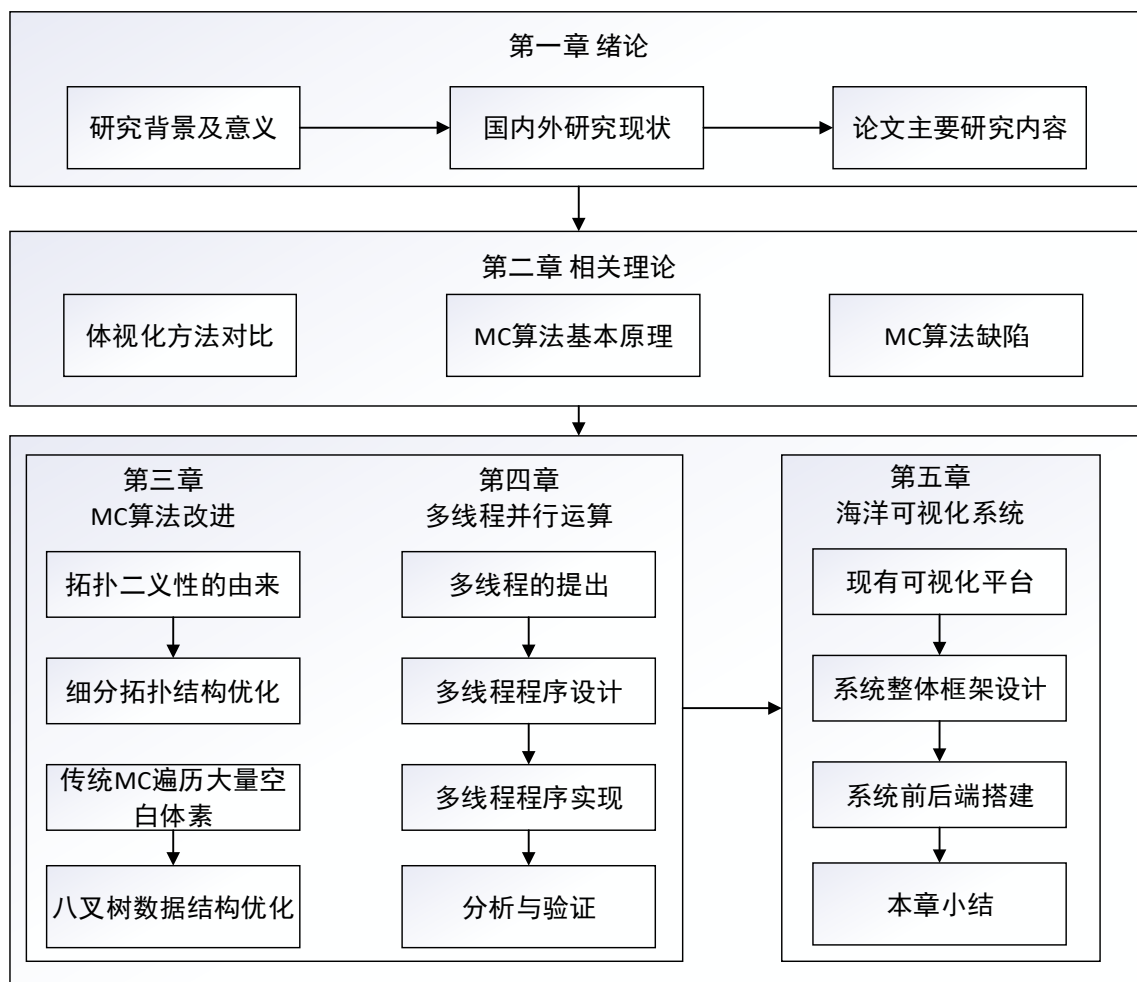


图 1.1 论文结构安排

第2章 体视化相关理论与方法

随着可视化领域的发展，体视化是一门庞大的技术，有很多技术能够实现对三维数据的描绘，但每一项技术都各有利弊。本章对现有体视化方法进行总结对比，并结合海洋体数据场进行分析。选出最适合对海洋数据进行可视化绘制的 MC 算法，对其原理进行介绍，归纳了原有算法的不足并指出其优化方向。

2.1 体视化介绍

体视化技术是将三维的场数据以图像等形式渲染出来，由于核磁共振技术、电子计算机断层扫描技术（CT）等医学领域的相关技术在 20 世纪 70 中仍然是二维图像可视化技术，当时人们迫切需要一种能够三维显示人体器官组织以及病变部位的可视化方法，于是在 20 世纪 80 年代提出体绘制的相关概念。

体绘制（volume rendering）也称体渲染，当时对于体的定义并不明确，但大概可以描述为：体视化技术是将三维的离散数据集投影为二维图像，并以此显示出来的技术。Levoy M 认为体绘制需要不仅仅限于展示“表面细节”，也需要展现一定的内部构造。他将体绘制定义为：一类从 3D 标量数据中生成 2D 图像的方法。从这一点出发，本文认为体渲染技术的关键在于显示出其内在的信息，而不仅仅是显示其表层的详细信息。所以，体视化就是对 3D 图像进行解析，并在 2D 图像上实现内部和外部的全部信息。当 3D 模型生成过程中，若生成中间几何单元，则应将其删除，最后得到的不只是外形结构，而是整个 3D 模型的外观。

常见的体视化技术应用如 CT、MRI 或 PET 技术都是将三维数据场中的信息提取，处理后重建为二维图像，但得到的图像是拥有内部信息的切片图像，与照相机拍摄得到的二维图像有着本质区别。照相机拍摄得到的二维图像仅仅含有物体的表面信息，无法看清物体的内部结构，而 CT 仪从三维数据场中获得数据，展示的二维图片包含着物体的内部数据。例如，磁共振技术产生的图片展示的是患者脑部或胸部等内部图像，而不仅仅是表面信息。综上所述，体视化技术的最终目标是通过提取利用三维空间数据场，将三维细节在二维图片上展示。举例而言，当一个飞机模型被拍摄得到图片时，只能得到该模型的表面信息，无法透过表面观察内部，而体视化技术绘制成的飞机模型可以从外到内观察到模型的所有细节特征。

体视化技术早期主要应用于医疗领域，医学领域需要高分辨率低噪声三维图像，该

需求推动体视化的发展，在体视化技术得到发展后，越来越多的领域开始应用体视化技术，如分子构造领域、地质学领域、海洋与气象领域等。相比于医疗领域，海洋气象领域应用体视化技术主要目的是强化视觉效果，将很多视觉上不规则的自然物体进行绘制，如云、烟、海温层等等。由于它们不规则的特性，利用物体建模手段很难对他们进行描绘，有学者利用粒子系统进行模拟，但是效率和效果都欠佳，而体视化技术能够快速、准确的将海洋气象领域要素绘制出来。

除体视化外，该技术还有别的名称，如体绘制和体建模等。体视化全名为体可视化，是该技术的统称，包含所有对三维数据绘制的各种方法。体建模是利用体数据构建表达三维物体和现象模型的过程，体绘制是指将 3D 的离散采样数据集通过 2D 投射显示出来的技术。体视化是一门庞大的技术，有很多技术能够实现对三维数据的描绘，但每一项技术都各有利弊，如何选择一项合适的技术用来对海洋相关数据进行体视化研究是一个重要的问题。

2.2 三维可视化方法分类

海洋观测数据按其性质可划分为标量场数据、向量场数据及张量数据。按照数据表达方式及渲染方式的差异，体视化可以划分为基于面绘制的体视图（SBR）和基于体绘制的体视图（DVR）。由于科技的发展和要求的多样化，这两个方面的研究也逐步发展出许多更加精细的算法。面绘制主要包括：轮廓线连接法、行军立方体法（MC）、移动四面体法、剖分立方体法等。直接体绘制又分为两大流派：空间域体绘制和转换域体绘制。其中，空间域体绘制技术主要包括图像体绘制、物体序列绘制、混合序列绘制等；转换域的体绘制技术主要包括：基于压缩域的体绘制、频域绘制和小波域体绘制。体绘制全称是直接体绘制，它对三维图像进行解析，以一个单独的物体为基础，产生一个立体的体渲染视图。直接体绘制技术的优势在于其所产生的图像可以很好地反映物体的内在结构，但存在运算量大、运算时间长、效率低等问题；而面绘制技术拥有相对较快的运算效率，但绘制出的视图是由曲面构成的，无法直接观测物体的内部结构。除去单纯的面绘制与体绘制之外，还存在将两种绘制方式混合使用的混合绘制方法。图 2.1 为立体视觉的类别示意图。

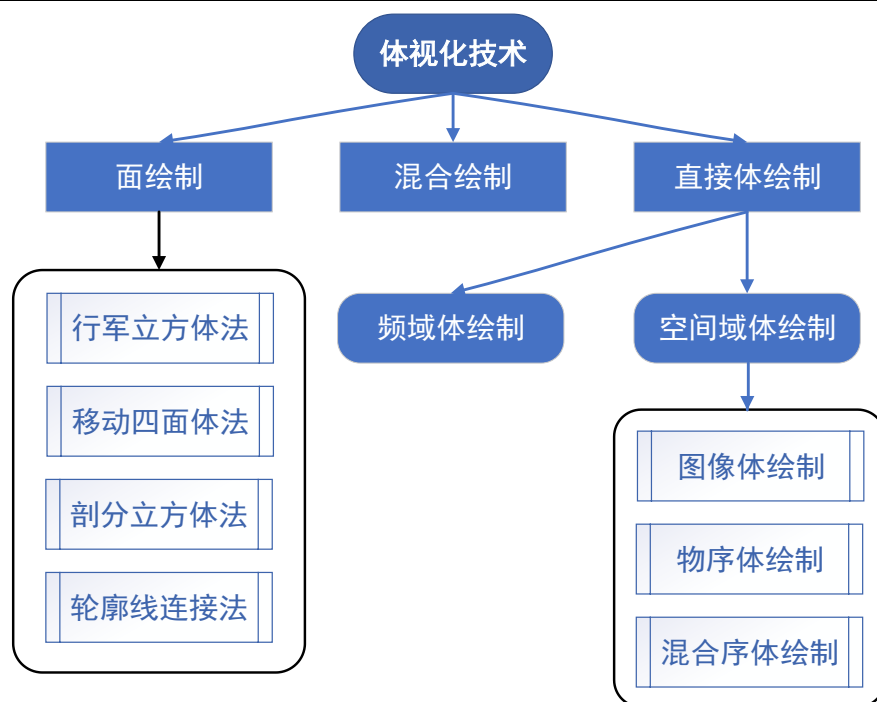


图 2.1 体视化技术分类

2.2.1 直接体绘制

直接体绘制技术主要分为空间域和转换域两大类型，它们都是在 2D 画面上投影 3D 数据^[50]，无需构建任何中间的几何要素，能够产生完整的 3D 数据场图像，同时也包含体数据的内在信息，具有较高的渲染品质和更好的深度。但是该算法的运算量非常庞大，并且难以通过常规的图形化软件来实现，从而造成大量的运算时间。

直接体绘制方法是一种用于可视化三维数据场的技术，它可以将数据直接投射到二维屏幕上，而无需构建中间的几何元素。这种方法可以提供高质量的整体图像，包括体数据内部的信息，并能够呈现数据的深度信息。直接体绘制方法可以分为两大类：空间域体绘制和变换域体绘制。在空间域体绘制中，三维数据场中的每个体素直接映射到屏幕上的像素点，通过对每个体素进行绘制，最终形成完整的三维图像。而在变换域体绘制中，通过使用数学变换和算法，将三维数据场转换为频域或纹理坐标，然后将其投射到屏幕上进行绘制。能够显示完整的三维数据场，包括内部信息和深度信息。然而，直接体绘制方法也存在一些挑战。由于计算量巨大，需要大量的计算资源和时间。这导致计算时间较长，不适合实时应用。直接体绘制方法很难利用传统的图形硬件进行加速，限制其在实时应用中的使用。此外，直接体绘制方法在处理大规模数据时也会面临内存和存储的挑战。为克服这些问题，研究人员一直在不断改进直接体绘制方法。他们通过优化算法和利用并行计算等技术，尝试减少计算时间和提高绘制效率。同时，还在探索

新的硬件加速技术，如图形处理单元（GPU）和专用的计算加速器，以提供更快的计算速度和更高的绘制性能。直接体绘制方法是一种重要的三维数据可视化技术，它能够呈现完整的三维数据场，并提供深度信息。

（1）空间域体绘制

空间域体绘制方法的实质是通过重采样和颜色合成来实现对三维数据场的可视化。按照绘制顺序，将其划分为三种类型：图像序列体绘制、物序体绘制和混合序体绘制。图像序列体绘制是一种基于图像序列的图像渲染方法，该方法首先从图像中提取出图像中的图像，然后根据图像中的图像信息对图像进行处理。物序体绘制是将物体的体积信息投影到 2D 画面上，通过对各像素的作用来实现物体的渲染。此外还有基于图像序列与物体序列的混合序列绘制方法。各种方法在实际情况下都有其优缺点。

1) 图像序列体绘制技术^[51-54]：利用射线在数据区中的传播来判断图像中各像素的色彩大小的绘制方法。其基本特征是：从显示屏上的各个像素开始，按照观察点的方位将射线通过 3D 体数据区，然后再将其分散。然后利用该方法得到各样本点间的色彩及不透明度，进而得到相应的画面像素的色彩。目前常用的图像序列体绘制技术主要有 X 射线法、最大灰度投影法和等值面法，这三种算法都是通过对图像样本的采集和处理，从而得到图像的彩色信息。例如，将插值样本点的灰度值用作像素的色彩，而最大亮度投射规则将插值样本点的最大值用作像素的色彩。

2) 物体序列绘制算法^[55]：将物体数据投影到 2D 画面上，从而实现物体的渲染。该算法通过对各像素点在显示屏上的作用来判断各像素点的色彩。在此基础上，提出基于网格空间坐标的体像素点的映射方法，计算目标区域的影响范围，给出基于 Splatting 的网格生成方法。

3) 将图像序列体绘制与物体序列体绘制相融合的混合序列体绘制方法。该方法将空间域的投影转换问题分为 Shear 和 Warp 两种类型，从而大大降低采集时间。该算法可以近似于实时地进行体视图，而不会显著地影响到图像的质量。该方法以简单的 2D 抽样为代价，在保证成像效果的同时，加快渲染速度，如果对画质要求不高，可以考虑使用混合序列绘制方法。

（2）变换域体绘制

变换域体绘制方法将空间体数据转换到变换域，然后利用变换域的信息生成投影进行显示^[56]。常用的体绘制领域包括压缩域、频率域和小波区域，相应的体绘制技术主要有压缩域体绘制、频域体绘制和小波体绘制。这些方法利用变换域的特性进行可视化和显示，提供不同的方式来呈现体数据。

1) 基于压缩域的体绘制技术是在不经过任何解压的情况下, 将已被压缩的数据在压缩域中进行渲染。无需加载所有未经压缩的体积资料, 绘图的效果即可即时呈现。该算法在保证图形渲染精度的前提下, 减少对计算机存储、运算和传输等方面的需求, 提高渲染的效率, 并将渲染和数据压缩相结合。首先, 采用矢量量化的方式对其进行空域的体积图像压缩, 并采用通用的空间体渲染算法实现图像的快速渲染。该算法不仅提高渲染效率, 而且提高渲染速度。

2) 频率域体绘制方法^[57]: 利用傅里叶分片原理, 将体积渲染应用到频率域中。利用傅里叶分段原理, 对空间中的二维空间进行傅里叶变换, 提取出与视野方向相垂直的二维剖面。对该断面作反傅里叶转换, 即可获得空间域内像面的投影。频率域体渲染方法是通过对 3D 数据进行傅里叶变换, 获得在频率域上的数据域表达式。对经过该坐标轴的、与该投影方向相正交的平面上的二维断层重新取样, 在此基础上, 对两维采集的流场做二维反傅里叶变换, 可以得到两个平面上并行的投影图像。

3) 小波域体渲染是将体积数据用小波进行 3D 的离散化处理, 以达到多分辨表达与压缩的目的。在渲染过程中, 将小波的系数替换到体画公式中, 不需要对其进行解压, 即可得到 2D 图像。小波分析因其局域特性, 能用少量的小波系数来表达体积资料。

(3) 方法小结

直接体绘制及其相应特点总结为表 2.1。

表 2.1 直接体绘制与相应特点

具体算法	图像质量	绘制速度	算法特点
3D 纹理映射	较低	较快	内存占用大, 可以用硬件加速
错切变换算法	一般	极快	内存占用小
光线投射算法	极高	很慢	内存占用大
足迹追踪算法	高	较快	内存占用小, 可渐进显示
频域体绘制	较高	快	算法简洁, 能达到 X 光效果
基于小波体绘制	高	快	内存占用大, 有整体效果

2.2.2 面绘制

面绘制的特点是针对体表面进行重建, 从三维空间数据场中寻找最接近所求值的三角面片或多边形来组成等值面。数据场中通常不存在完全吻合所求值的点位, 通常依靠各种方法来得到逼近该值的点位, 然后利用周边信息计算渲染所需相关数据。面绘制方法简单, 可以结合计算机硬件的发展对方法进行优化加速, 大大降低渲染所需用时。速率快, 方法流程简单, 产生的图像清晰度高, 但是对体数据内部的信息展示有所欠缺,

无法展示全局信息。常见的面绘制方法有：轮廓线连接法、移动立方体法、移动四面体法和剖分立方体法四种。

（1）轮廓线连接法

轮廓线连接法通过提取每层图像的轮廓线、选择轮廓线上的顶点、连接顶点形成三角面片，并通过光照计算进行渲染得到三维图像。只适用于层间等值面变化较小、几何形状相似且绘制精度要求较低的场景。

（2）行军立方体法

行军立方体方法是一种经典的面绘制重建方法。该方法通过将体素内部的 8 个相邻体素构建的三角面片表示为等值面，从而对等值面进行构建。它能够处理规则体数据场，并能够准确地绘制等值面。然而，由于面片数量较多和二义性问题，该方法在处理复杂数据场时可能会面临挑战。为了克服这些问题，研究人员已经对移动立方体方法进行了改进和优化，使其在不同应用领域得到了广泛应用。

（3）移动四面体法

移动四面体法是一种用于解决移动立方体法二义性问题的有效方法。通过对立方体体素进行剖分，将其分解为四面体，然后在每个四面体内部构建等值面，从而提高等值面重建的准确性和精度。

（4）剖分立方体法

剖分立方体法与移动立方体法类似，都是对数据场中的体素逐一进行处理。当体素顶点分别位于等值面内外时，沿着不同方向对体素进行分割，直到均处于等值面某一边。通过使用剖分立方体法，可以更准确地呈现高密度的数据场，减少了小面片的插值计算，提高了等值面的精度和效果。这种方法在处理复杂的数据场时尤为有效，可以有效地克服移动立方体法在高密度场景下可能面临的问题。

2.3 海洋体数据场

2.3.1 体素的介绍

体素（Voxel）是体积元素（Volumetric pixel）的简称，是三维空间中数字数据分割的最小单位，类似于二维空间中的像素。体素在立体渲染和提取等值面等应用中起着重要作用。它可以用于表示三维成像、科学数据和医学影像等领域。每个体素可以被视为一个小方块，没有绝对的空间位置，只存在相对位置的概念。体素在数字数据的表示和处理中扮演着关键的角色。

通常规则三维体数据由 $j \times k \times l$ 个体素组成, 意味着 X 轴方向存在 j 个体素, Y 轴方向存在 k 个体素, Z 轴方向存在 l 个体素, 三个维度构成的整体则是该三维体数据。通常体素与体素之间的间隔是均匀且可知的, 如图 2.2 所示。例如 ERA5 的海洋数据资料在 X 轴的间距是 0.1 个经度, 在 Y 轴上是 0.1 个纬度。体素一般被看作是三个平面上四个邻接的网格点所构成的正方体。

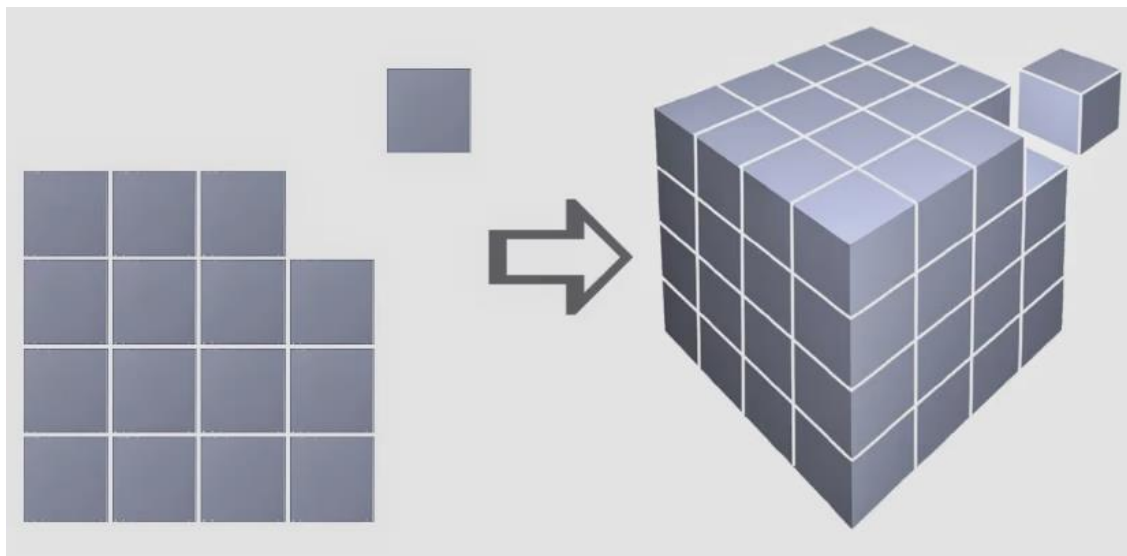


图2.2 平面体素与空间体素

体素数据由一个均匀的、有规律的网格组成, 网格中的点之间互相联系构成一个六面体(如立方体形)。均匀网格具有其数据结构简单、存储空间紧凑、以及高效的查询等优势。然而, 均匀网格的适用性略有不足, 所以当需要表示离散的数据时需要采用形变六面体等非均匀网格, 或是棱镜等非网格数据格式。

2.3.2 体数据场

从 2.3.1 节可知, 体视化是通过提取利用三维空间数据场^[58], 将三维细节在二维图片上展示。三维空间数据场是由一系列采样点数据构成的集合, 这些采样点被称为体素。采样点数据通常包含基本信息 x, y, z, v 。其中 x, y, z 分别代表空间坐标系中三维位置坐标 (x, y, z) , 而 v (value) 代表该位置所具有的值, 该值可能不止含有一个。同时, 由于值的代表意义不同, 可以将体数据场分为二值数据场、标量数据场、向量数据场和时变数据场四类。当采样点值 v 只存在 0 与 1 两种情况时, 该体数据场为二值数据场; 当 v 的值不止 0 和 1 两个数, 并且所代表的含义是只有大小没有方向的标量性质时, 该体数据场为标量数据场, 如海温, 海盐浓度, 海水密度等; 当 v 的值存在不止一个, 且其中一个代表方向时, 该体数据场为矢量数据场, 如声速, 海流, 风场等; 当采集点

的数据存在 x, y, z, t, v 五个维度时, 证明该点位的数据是会根据时间而变化的, 其中 t 代表具体某一时刻, 在这一时刻下检测到变量值的大小为 v , 该体数据场为时变数据场。

随着传感器技术的不断进步, 人们现在能够获取到越来越多的海洋数据, 这些数据通常是四维的采样点数据, 包含时间和空间信息。除此之外, 根据采样点在空间中的分布情况, 可以将体数据场分为规则数据场和散乱数据场两类。在规则数据场中, 数据点的空间位置非常均匀, 这种数据场包括笛卡尔型、等距型和矩形型三种类型。这些数据场中的体素都是六面体的形状。散乱数据场的点位分布毫无逻辑, 没有规则, 用散乱数据场数据来进行体视化可以先通过几何方法转化成规则数据场, 此时体视化的本质仍然是使用规则数据场。

现有的海洋环境数据大多以 NetCDF 文件格式进行存储, 该文件格式以 .nc 为后缀。最初设计用于存储气象领域, 由美国大学大气研究协会开发, 在后续的发展下已经成为很多数据采集软件使用的文件格式。NetCDF 文件通常会将数据的各个维度作为标签独立存储, 例如海洋温度要素中会将包含某个区域海洋温度的经度, 纬度, 高度(或深度), 时间等标签。

2.4 传统 MC 算法

2.4.1 传统 MC 算法的原理

MC (Marching Cube) 算法, 也叫行军立方体算法, 该算法于上世纪末被 W.Lorensen 等人提出, 由于该方法的理论简洁、容易理解, 便于编程, 因此得到越来越多的使用。算法本身的思路是假设离散数据场内部的数据变化是连续的, 找到等值面经过的体素网格, 并计算出体素的边与等值面的交点, 最后将交点按设定好的拓扑关系进行连接得到等值面。等值面是三维空间中的等值线, 是将空间中某个相同数据的点位组合而成的连续的曲面。它可以表示为:

$$\{(x, y, z) | f(x, y, z) = v\} \quad (2-1)$$

其中, x, y, z 是空间中某个点位的坐标, v (value) 是设定好的所求等值面数值。

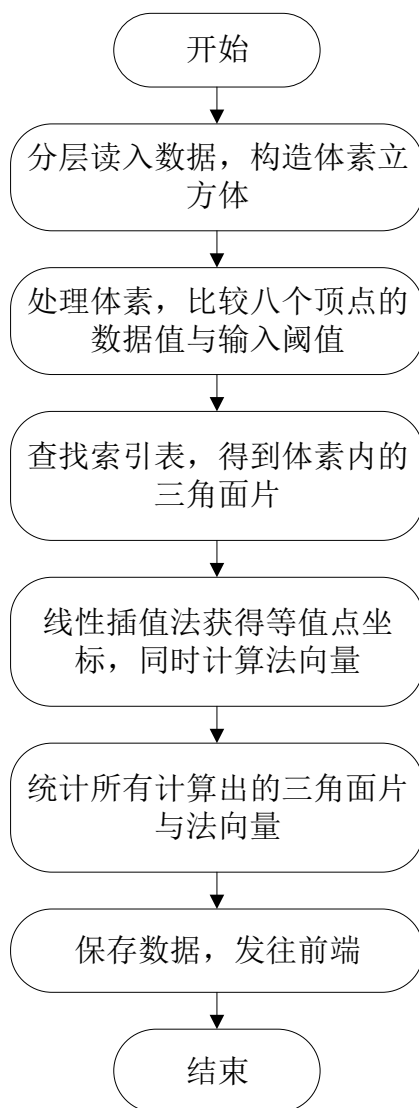


图 2.3 传统 MC 算法流程图

等值面提取方法大致可以分为两种，直接提取和间接提取。直接提取方法是对三维数据进行处理，得到连续函数，以此来比较设定的阈值，从而形成等值面，具体流程如图 2.3 所示。这种方法需要对数据场进行插值或重建，以获得连续的函数表示。然后，通过比较函数值和阈值，确定等值面的位置和形状。这种方法在计算上相对较为复杂，但可以得到精确的等值面结果。间接提取方法则是直接从离散的采样数据场中提取信息，而不需要重构连续函数。该方法不直接计算函数根据阈值抽取等值面，而是通过分析数据点的位置和数值来确定等值面的位置和形状。这种方法相对于显示提取方法更加高效，适用于大规模数据集和实时应用。

在一个立体的栅格环境中，每个采样点可以被视为位于相邻栅格之间的位置。栅格之间的距离几乎接近像素的尺寸，这样可以确保数据场的连续性和准确性。在这样的栅

格环境中，相邻两层栅格中的每 8 个顶点可以组成一个立方体，也称体素，如图 2.4 所示。这是因为在三维空间中，立方体的八个顶点可以通过连接相邻的栅格顶点来定义。这种立方体的形式也被称为八个顶点规则（Eight-Vertex Rule）。通过将采样点与立方体栅格相对应，之后可以使用 MC 来提取等值面。通过对每个立方体进行阈值判断，确定是否存在等值面，并通过插值方法来计算等值面的位置和形状。这样就可以将离散的三维数据场转化为连续的曲面表示，从而方便进行可视化和分析。

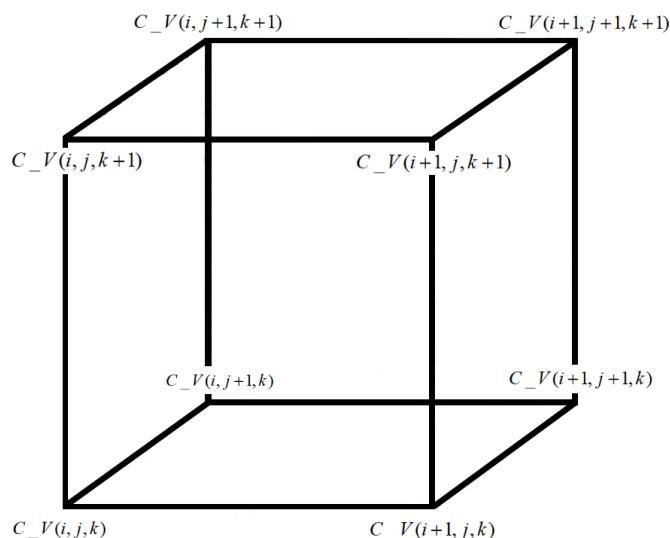


图 2.4 体素坐标示意图

假设体素直接的每两个数据点都为均匀变化，此时体素内的随机点都能够针对物理坐标使用线性插值的方法计算出值的内容，设点的位置为 $C_V(x, y, z)$ ，坐标为 (i_6, j_6, k_6) ，其中， $i_3 = x / \Delta x$ ， $j_3 = y / \Delta y$ ， $k_3 = z / \Delta z$ ，则经过线性插值得到的体素数据为：

$$f(C_V_6) = f(C_V_4) \times (i+1-i_6) + f(C_V_5) \times (i_6-i) \quad (2-2)$$

$$f(C_V_4) = f(C_V_0) \times (j+1-j_6) + f(C_V_3) \times (j_6-j) \quad (2-3)$$

$$f(C_V_5) = f(C_V_1) \times (j+1-j_6) + f(C_V_2) \times (j_6-j) \quad (2-4)$$

$$f(C_V_n) = f(i(n), j(n), k) \times (k+1-k_6) + f(i(n), j(n), k+1) \times (k_6-k) \quad (2-5)$$

式 (2-5) 中：

$$i(n) = \begin{cases} i, & n=0,1 \\ i+1, & n=2,3 \end{cases}, j(n) = \begin{cases} j, & n=0,3 \\ j+1, & n=1,2 \end{cases} \quad (2-6)$$

将式 (2-5) 和式 (2-6) 整理可得：

$$f(x, y, z) = a_0 + a_1x + a_2y + a_3z + a_4xy + a_5yz + a_6xz + a_7xyz \quad (2-7)$$

由式(2-7)可以发现,在假设前提下可以利用体素顶点的数据计算出内部任何一个点的数据值。当算法读入三维海洋要素数据时,将每相邻两层切片8个顶点组成立方体体素,若体素中的棱边两个顶点的数据值一个大于所求阈值,另一个小于所求阈值,则判断该棱边上存在等值面的交点,称为等值点。由于使用MC算法绘制的海洋数据往往是标量数据场,因此采用线性插值来计算具体的等值面坐标。对比八个顶点与所求等值面阈值,若顶点的数据值大于所求阈值,则将其标记为1,说明该点位的位置在等值面的外侧;若顶点的数据值小于所求阈值,则将其标记为0,说明该点位的位置在等值面的内侧。一个体素共计8个顶点,每一个顶点存在0和1两种情况,所以每个体素共计 $2^8 = 256$ 种状态。但由于立方体有着中心对称和左右对称的特点,此外还存在指置反顶点标记的对称性,即所有点位同时进行01互换。通过对称性,可以将256种状态压缩归纳成128种状态。再结合立方体中心对称和左右对称,最终能够归纳出最为精炼的15种基本构型,简化后的基本构型如图2.5所示。

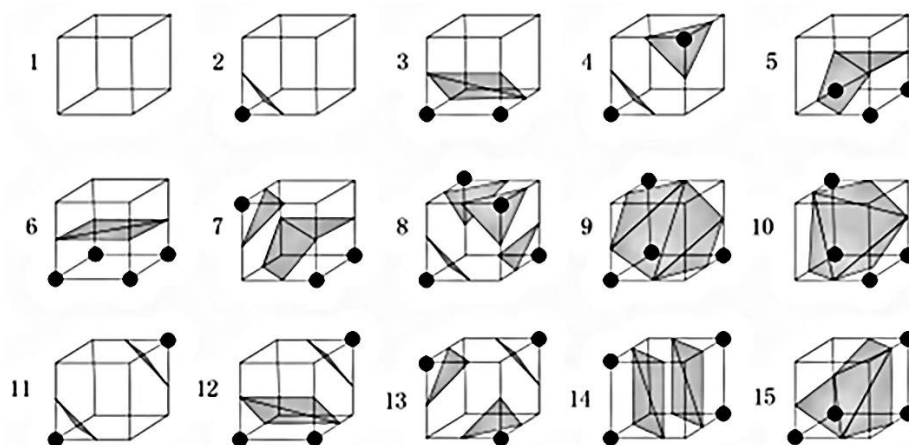


图 2.5 等值面体素的 15 种构型

在总结MC算法的15种基本构型之后,为方便后续场合的描述与说明,需要将体素的每个顶点和棱边进行命名编号。顶点编号根据体素的位置来具体对应,靠内的左下角为编号0,并沿同一层顺时针编号增大,上一层同理增加,直至左上角外侧,编号为7。而程序中用八位二进制数来表示索引情况,根据八个点对应的编号以及点位数据是否大于所求阈值对二进制的位数进行赋值,赋值后的顶点索引可以表示为: Index = 7 6 5 4 3 2 1 0, 将八位二进制 Index 转化为十进制数后范围在0~256之间,对应所有共计256种情况。为将内存的占用再度进行压缩,转化为十六进制之后再存储。棱边索

引则按照 X, Y, Z 三轴的顺序进行依次命名, 表示为 $\text{Edge} = 11\ 10\ 9\ 8\ 7\ 6\ 5\ 4\ 3\ 2\ 1\ 0$ 。顶点索引和棱边索引在体素中的位置如图 2.6 所示。

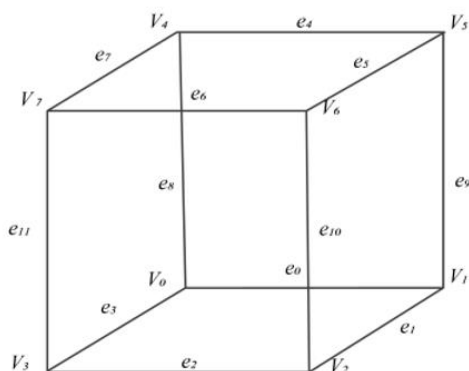


图 2.6 索引示意图

除去顶点索引和棱边索引, MC 算法还有一张三角面片索引表, 该索引表包含所有 256 种情况中点的索引表对应的具体三角形面片的组合方式。通过该索引表, 能够将每一个体素内部所对应的三角面片情况的反复计算, 而直接进行查表获取, 降获取面片步骤复杂度降低为 $O(1)$ 。前文中假设体素所有的点都是均匀变化的, 通过三线性插值能够获得内部任何一个体素点的数据值, 因此棱边中的数据值也能够用相同方法计算, 方法如下:

(1) 遇到与 X 轴平行的棱边时, 设定棱边距离 Y 轴的点为 $C_V(x_1, y, z)$, 则另一点为 $C_V(x_2, y, z)$, 将棱边上存在的等值点坐标设置为 $C_V(x, y, z)$, 通过将顶点 1 的 I_1 和顶点 2 的数据值 I_2 代入公式 (2-8) 可以计算出 x 的数据:

$$x = x_1 + \frac{(T - I_1)(x_2 - x_1)}{I_2 - I_1} \quad (2-8)$$

(2) 遇到与 Y 轴平行的棱边时, 设定棱边距离 X 轴的点为 $C_V(x, y_1, z)$, 则另一点为 $C_V(x, y_2, z)$, 将棱边上存在的等值点坐标设置为 $C_V(x, y, z)$, 通过将顶点 1 的 I_1 和顶点 2 的数据值 I_2 代入公式 (2-9) 可以计算出 y 的数据:

$$y = y_1 + \frac{(T - I_1)(y_2 - y_1)}{I_2 - I_1} \quad (2-9)$$

(3) 遇到与 Z 轴平行的棱边时, 设定棱边距离 X 轴的点为 $C_V(x, y, z_1)$, 则另一点为 $C_V(x, y, z_2)$, 将棱边上存在的等值点坐标设置为 $C_V(x, y, z)$, 通过将顶点 1 的 I_1 和顶点 2 的数据值 I_2 代入公式 (2-10) 可以计算出 z 的数据:

$$z = z_1 + \frac{(T - I_1)(z_2 - z_1)}{I_2 - I_1} \quad (2-10)$$

2.4.2 传统 MC 算法存在的问题

MC 算法是一种经典的面绘制算法，通常用来绘制医学图像等等，通过遍历体素，获得每个体素内的三角面片，通过使所有三角面片相互连接来组成三维立体视图。具有计算效率较高，流程简单等优点，适合数据量庞大的可视化系统使用。由于计算机硬件的更新，MC 算法也能够结合各种计算机硬件，例如 CPU 的迭代开始往多核心方向进行转变，MC 算法也能够做出相应的优化。在医学领域与地质学领域中，MC 算法得到大量的应用与针对性的改进，但对于三维海洋可视化方面，MC 算法缺乏很多针对性的研究。除此之外，传统 MC 算法也有很多内在缺点，需要进行改进，主要的改进方向分为三类：立方体的拓扑二义性造成面片位置信息错误，体现在具体图像上为等值面产生“空洞”；遍历体素进行计算导致计算量大；逼近精度不够高。

针对第一类改进方向，当两个面相邻的体素的重合面存在面二义性时，存在产生二义性的可能，空间二义性的根源来自面二义性，是无法避免的逻辑原因。当出现拓扑二义性错误时，原本连续的等值面会由于错误的拓扑连接关系而产生“空洞”。二义性的具体原因本文后续会进行深入分析。

针对第二类改进方向，传统 MC 算法本身需要遍历所有立方体，而往往全部的立方体体素中只有少量体素内存在需要的三角面片，但往往绝大部分的体素都是空白的，这会导致大量的时间用来计算无效体素。除此之外，计算机硬件的高速发展为 MC 算法的优化也提供方向。

针对第三类改进方向，对于绘制逼近精度不高的问题，算法本身采用的是通过线性插值的方式来获取三角面片与立方体体素的交点位置，通过将所有三角面片相连的形式获取三维整体视图。这种方式在实际使用中，整体精度已经足够，在数据量足够庞大时，甚至可以直接使用棱边的中点作为面片的顶点位置，以此来提高运算效率，绘制精度仍然满足要求。但如果传入的海洋要素数据精度较差时，绘制出的图像精度可能不足，此外若有需要对其中一小块区域进行放大观察等对精度要求较高的情况时，需要相应办法进行改进。目前常见的方法是增加内部肩点和使用渐近线网络两种方式。由于本文中海洋要素数据量相当庞大，没有提高逼近精度的需求，不再重点进行介绍。相应的改进方法需要根据具体需求进行选择。

2.5 MC 算法的优化方向

由上一节可知, MC 算法本身存在很多缺陷, 例如拓扑二义性无法保证, 遍历大量无效体素导致效率低下等等。MC 算法流程简单, 可以结合计算机硬件的发展对方法进行优化加速。这些都是需要优化的地方, 除此之外, 结合海洋三维数据场的特点来进行针对性优化也是一个方向。

(1) 解决拓扑二义性问题

二义性的产生源于其本质的面二义性, 当对角的两个体素点的所含值都大于所求阈值时, 此时面的所有棱边都含有阈值点位, 四个点位之间的连接方式存在着两种方式, 这就是二义性的本质由来, 解决二义性的方法有很多, 可以针对能够出现二义性的拓扑情况进行针对性的划分, 也可以采用双曲线渐近线等方法, 获取周围体素的信息, 以此来推定当前体素的面片位置。二义性影响的是体视化的准确性, 因此是优化的重点内容。

(2) 优化 MC 算法效率

全球绝大部分区域都是海洋, 因此海洋要素的数据量十分庞大。由于海洋三维数据场有着数据量庞大的特性, 优化 MC 算法的效率尤为重要, 传统 MC 算法需要遍历全局每一个体素, 其中包含大量的空白体素, 如果能将空白体素单元剔除, 将会对算法带来非常大的优化。

(3) 多线程并行处理

面绘制可以结合计算机硬件发展进行优化加速, 近十年来计算机在 CPU 多线程领域以及 GPU 并行加速领域发展十分迅速, 如果能巧妙利用上述两项技术, 能够大大缩短绘制等值面的所需时间。

2.6 本章小结

本章仔细分析体视化现有的理论方法, 通过分析对比所有体视化方法的优缺点进行比较, 选出最适合对海洋要素进行可视化的 MC 算法。之后对海洋体数据场进行介绍, 对海洋数据的存储形式与特点进行了说明。对传统 MC 算法的原理进行阐述, 并对 MC 算法存在的问题进行了分析, 针对其各项缺点提出了算法的优化方向。

第3章 MC 算法的改进

由第2章可知,传统MC算法有很多方向需要改进优化,主要分为保证等值面拓扑正确性、提高算法效率、减少存储空间、提高逼近精度四个方向。四项改进方向中,最为核心的保证拓扑二义性方面。确保绘制的正确性是算法的基础。除此之外,传统MC算法本身需要遍历所有立方体,而往往全部的立方体体素中只有少量体素内存在需要的三角面片,但往往绝大部分的体素都是空白的,这会导致大量的时间用来计算无效体素。在面对海洋数据中庞大的数据量,传统MC算法往往会导致绘制时间过长,需要针对性优化算法本身以提升算法效率。

3.1 拓扑二义性改进

3.1.1 空间拓扑关系

空间拓扑关系本质上是不同物体间的相对位置关系,用在三维体素中则是描述体素之间的相对位置^[59],本文将常用的立方体体素拓扑关系总结并进行简单分析。体素之间的拓扑关系分别是面相邻,边相邻和点相邻,上述三种相邻形式分别对应着共用面,共用边,共用点三种共用关系。如图3.1所示,其中(a)图为面相邻、(b)图为边相邻、(c)图为点相邻。

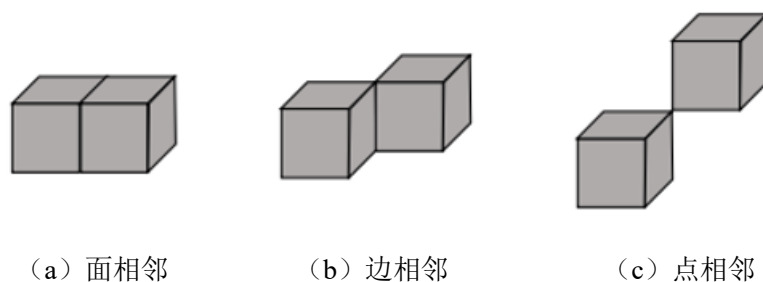


图 3.1 体素的相邻关系

由 $3^3 = 27$ 可知,一个体素一共存在 26 个相邻的体素,而一个体素有着六个面,十二条棱边,八个顶点,因此面相邻有 6 个,边相邻有 12 个,点相邻有 8 个。有学者对三维空间中的邻接关系中定义为 N-邻接 $N \in \{6, 18, 26\}$ 邻接关系,6 邻接则表示两个体素是面相邻;18 邻接对应着 6 个面相邻关系与 12 个边相邻关系;26 邻接则对应所有的相邻关系,即 6 个面相邻、12 个边相邻和 8 个面相邻。其中不难看出三种相邻方式为

包含关系，26 邻接包含着 6 邻接与 18 邻接，18 邻接包含着 6-邻接，这也对应着约束的强弱关系，其中 6 邻接>18 邻接>26 邻接。如图 3.2 所示，其中（a）图为 6-邻接、（b）图为 18-邻接、（c）图为 26-邻接。

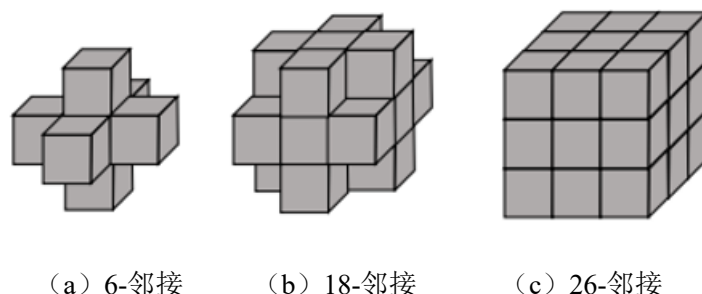


图 3.2 体素的邻接关系

3.1.2 拓扑二义性的由来

拓扑二义性的本身是在面对同一种数据形式有着两种不同的连接方式^[60]，其根源来自面二义性。面二义性是在一个体素的面上，若存在相对的两组顶点分别一组高于所求阈值，且另一组低于所求阈值时，存在正连接与反连接两种不同的线段连接方式。如图 3.3 所示。

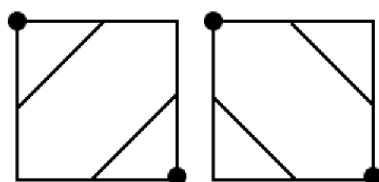


图 3.3 面二义性

当两个面相邻的体素的共用面存在面二义性时，若恰好两个体素分别使用不同的连接方式，此时面二义性出现。当面二义性出现时，且形成立方体后会产生体二义性，具体表现形式两个面相邻的体素的公共面上产生空洞现象。如图 3.4 和图 3.5 所示。

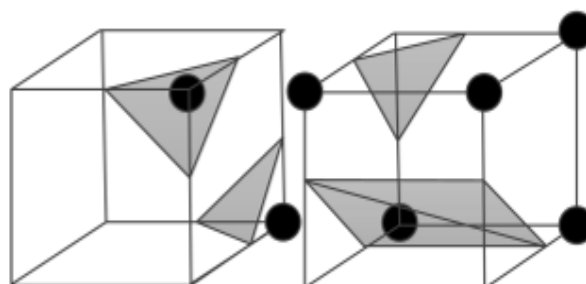


图 3.4 体二义性

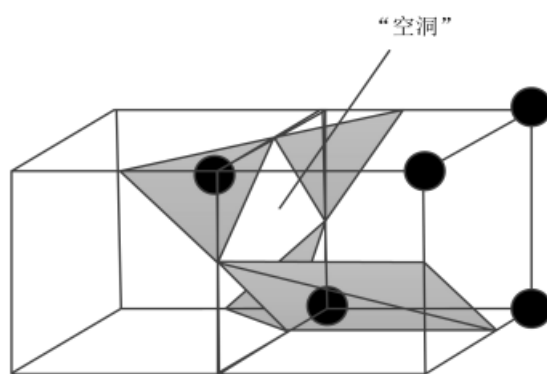


图 3.5 体二义性造成的空洞

3.2 MC 算法的拓扑正确性改进

3.2.1 改进方向分析

消除面二义性问题一直以来是 MC 算法改进的重点问题。消除二义性的关键所在是当出现面二义性情况时如何选择出正确的连接方式。目前主流的二义性解决算法分为四面体剖分算法和双曲线渐近线算法两类。

在利用四面体剖分方法进行二义性处理时，必须假定四面体数据的边界处没有断裂，而且全部数据为线性变化。由于四面体的每一个表面都是三角形，所以它们之间的联系也是独一无二的，从而在根本上解决二义性问题。根据海洋要素数据的分布特点来看，满足棱边数据场是连续变化，不会出现断层跨越现象，且也可视为是随着棱边线性变化的。但四面体剖分会导致生成的三角形面片数量大量增多，且会平白增加没有二义性时的体素的计算量，从而导致算法效率的显著降低。这与 MC 算法优化方向中的改进算法运算效率背道而驰，在庞大的海洋数据面前，四面体剖分算法不是适合的改进方向。

双曲线渐近线算法由 Nielson^[38]提出，由于等值面形状连续的特点，所以当立方体与等值面相交时，与其中一面的交线一定是某一对双曲线，若这对双曲线都碰到同一个立方体时，会出现两种连接方式。细分二义性情况，这对双曲线的渐近线交点都会在某一个区域，通过比较渐近线交点位置的数据与等值面的数据来判断拓扑连接情况。双曲线渐近线算法是目前较为主流的改进方式，但为保证拓扑正确性也增加额外计算开销。

对比上述两类方法，都能够很好的解决二义性的产生，但针对海洋领域中庞大的数据量，为兼顾算法效率，本文采用更加效率的细分拓扑结构方式进行优化。

由 3.1.2 小节可知，体二义性的来源是面二义性，当遇到二义性面时，会拥有两种线段连接方式，如图 3.6 所示。

设图 3.3 中左边的连接方式为正连接，右边的连接方式为反连接。当二义性面全部为正连接时，全部为反连接时，正反连接交替出现时和正反连接随机出现时，线条的连接方式如图 3.7 所示。从图像中可以看出全为正连接或全为反连接时，线条分布是规律的，即使正反连接交替使用，线条分布仍然具有规律性。只有正反连接随机使用时，线条变得杂乱无序。正反连接随机使用体现在二维空间中是线条的无序，当出现在三维空间时，会产生面片连接的错乱，体现在等值面上即为“空洞”。

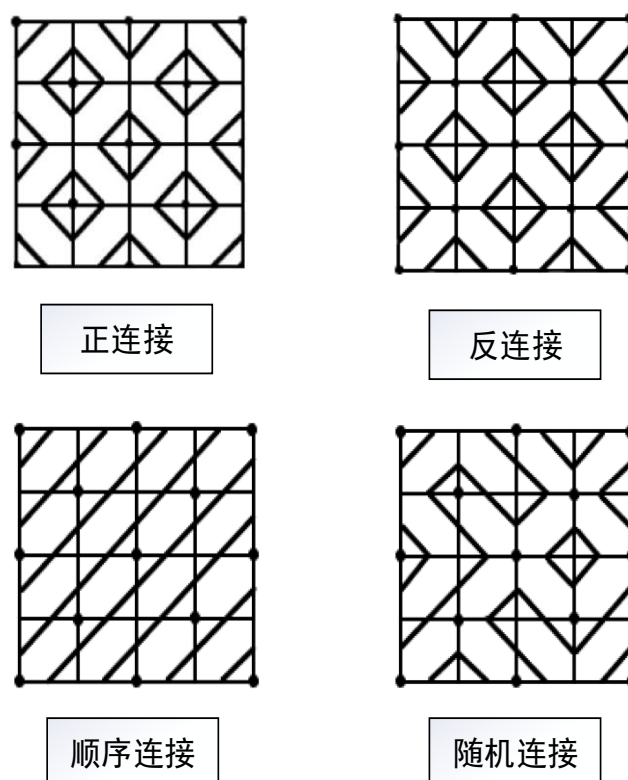


图 3.7 二义性面连接方式导致的区别

为避免正反连接随机使用的情况，本文对面二义性由 2.3 小节可知，MC 算法共有 15 种基本情况，但不是每一种情况都会产生二义性误区。首先对所有拓扑情况进行细分，对存在二义性面的拓扑情况进行进一步细分，对所有的二义性面设定为正连接方式，最终在原有的 15 种拓扑结构上增加 8 种拓扑分类结构，并给出对应的三角面片算法。如图 3.8 所示。

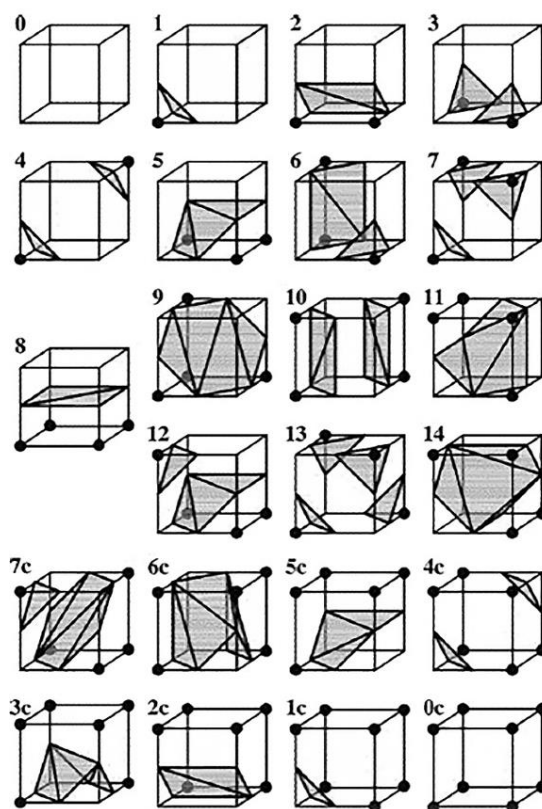


图 3.8 增加后的拓扑结构

通过强制设定正连接方式，得到 23 种新的基本构型。虽然其中部分构型能够被对称和旋转后统一归纳，但细分结构后能够在绝对保证算法效率的前提下保证等值面的连续性以及“空洞”的消除。但强制性的定义连接方式没有从图形的本身出发，考虑到海洋数据量庞大以及海洋要素等值面是连续渐变等特点，该方案是针对海洋可视化领域消除拓扑二义性最好的方式。

3.2.2 拓扑正确性验证

由 3.1 节可知二义性的产生是由面二义性的产生而产生，而并非每一次面二义性的产生都会出现“空洞”问题。梁秀霞和张彩明^[39]等研究者提出拓扑复杂度的概念，用于描述曲面的复杂程度。拓扑复杂度可以通过计算曲面上的临界点数量来衡量，在体元内部曲面的情况下，拓扑复杂度则是指体元内部曲面上的临界点数量。这种定义可以帮助研究者分析和比较不同曲面的复杂性，而拓扑复杂度最高的等值面结构为鞍面结构，可以认为，实际情况中，鞍面是几乎所有二义性产生的位置。为验证改进算法的正确性，本文挑选鞍面形状进行绘制分析，结果如图 3.9 所示。

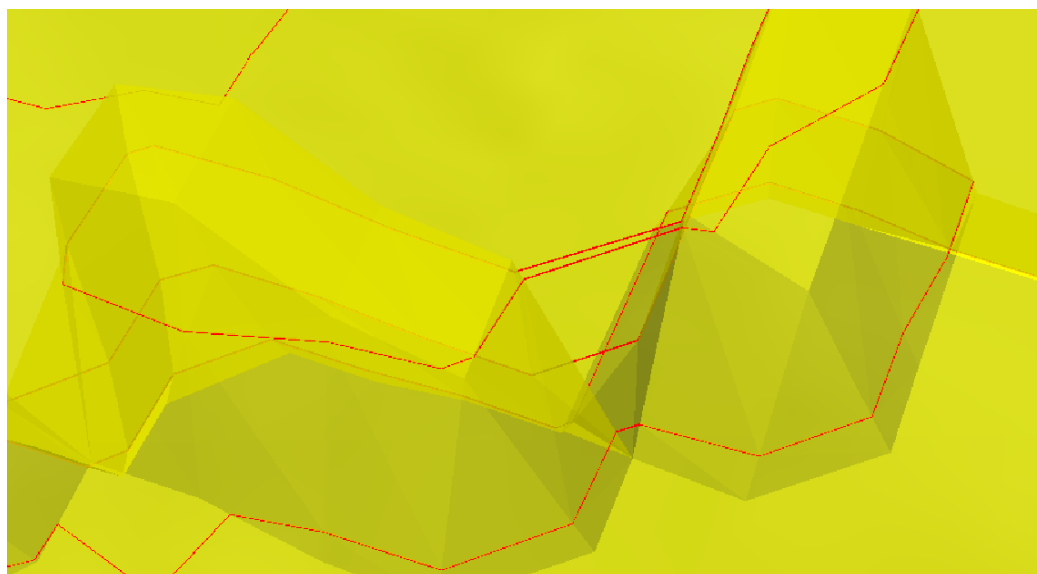


图 3.9 改进算法绘制鞍面

包围等值面的等值线因为面二义性产生错误的链接情况，但改进后的 MC 算法没有因为该部分面二义性而产生面片错误连接情况，使得较为复杂的“鞍面”正确绘制。

3.3 基于八叉树数据结构改进 MC 算法

八叉树是 N 叉树的一种，是根据空间的递推原则建立起来的一种层次式的数据结构^[60-62]，由一堆相互关联的节点组成。每一个节点都有一个父节点以及 0 个或 8 个子节点，没有子节点的节点称为叶子节点^[63,64]。二维空间中，存在着四叉树数据结构对二维平面模型的优化，拓展到三维空间中则为八叉树数据结构^[65,66]，该结构能够以递归的方式逐步分解一个完整的立方体，且大小立方体之间联系紧密有序，如图 3.10 所示。

图 3.10 中左侧为节点对应的空间立方体，右侧为八叉树数据结构中相应的节点位置，可以看出空间立方体被不断的细分，而细分到任何时候都有唯一的节点与之对应，通过层层划分直到找出空的体素节点或细分得到最小体素为止。八叉树数据结构通过在父节点中存储相似的信息，实现对数据的高效压缩^[67]。这种压缩方法可以减少存储空间的使用，并且在遍历和索引数据时具有较高的效率。因此，八叉树在许多模型的体素化过程中得到广泛应用。通过八叉树，可以更有效地处理大规模三维模型的数据，提高数据处理的效率和性能。在进行绘制等值面时，空白体素有时能达到所有体素中的 60% 以上，此外还有较为平坦区域，能够被合并绘制为较大的“块”。不论是合并绘制的部分还是空白体素，都能够大大提高算法效率、降低内存消耗。该方法被广泛地用于多种体素方法，从而提高数据的储存与处理效率，降低计算成本。

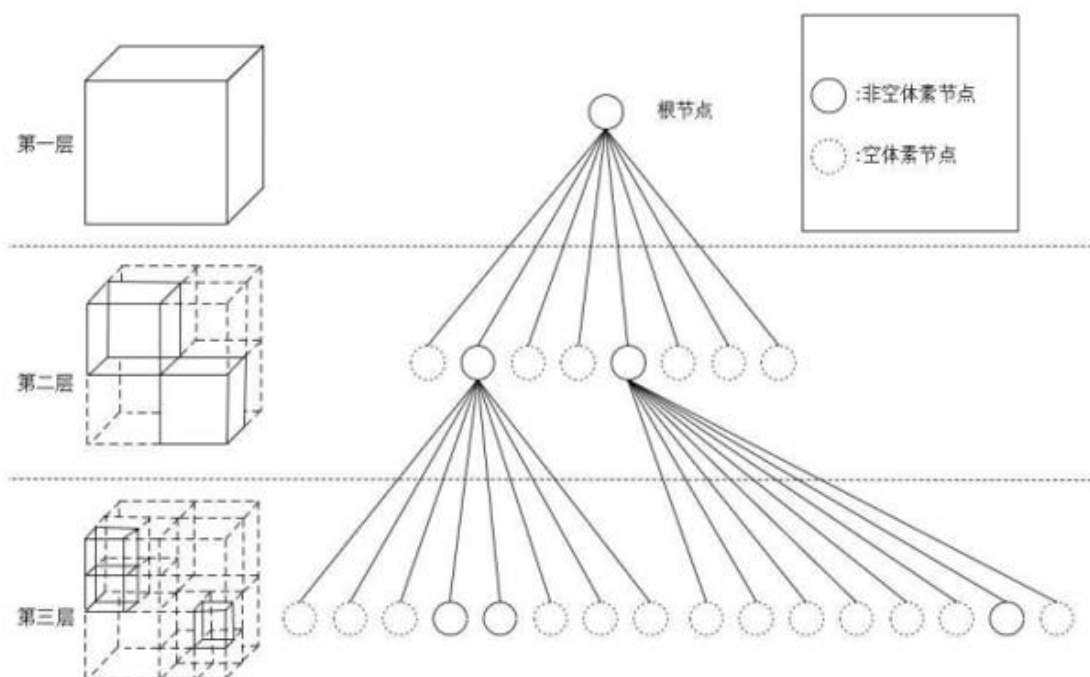


图 3.10 传统八叉树结构示意图

八叉树是一种树型的数据组织，它可以很好地刻画三维空间。每一个结点都是一个立方体，它可以包含 0 到 8 个子项。结点的容量是所有子结点容量的总和。其中，根结点代表整体的运算空间，而其它结点则代表该空间的子域。该方法能按照使用者的要求，将其分割成任何形式的网格。节块的厚度是表征网格单元剖分多少次的关键参量。其中，根结点的深度为 0，子结点的深度大于父结点的深度为 1，而同一个父结点的 8 个子项的深度是一样的。八叉树是一种常用的用于三维空间资料的方法，它能有效地进行数据的压缩与处理。

使用行程编码、稀疏分块网格、八叉树以及哈希表的局部水平集等方法，可以有效地减少原本算法的空间复杂度。八叉树算法是其中一种算法，它能对不同的边界进行有效的分割。这种方法仅需要保存用来剖分的各种节点，能够极大地减少对内存的占用。八叉树方法在水平集方法中具有重要的应用价值，能够有效地解决存储需求和提高计算效率的问题。八叉树算法最大的优点就是具有较强的自适应能力。目前，该方法已广泛应用于图像检索，图像处理，点云重建等诸多方面。对应三维领域中提出的八叉树结构算法，而在二维空间内，可以利用四叉树划分方法，实现对任意点的快速定位。在面向二维问题时，采用四叉树方法，可以使计算的时间复杂性降低到 $O(\log(n^2))$ ；而在面向三维空间问题时，采用八叉树方法可以将计算的时间复杂性降低至 $O(\log(n^3))$ 。这种自适应算法极大地提高搜索效率，并且能够在算法的具体实现中可以通过控制设定剖分的深度来控制精确性，深度越高精度越高，反之深度越低，精度也越低。

3.4 八叉树数据结构的程序实现

八叉树是一种采用递归思想的数据结构，将节点分解为八个子节点。它具有简单的存储形式和便于计算的优点。八叉树常用于对三维空间数据进行建模和处理。每个节点可以表示空间中的一个区域或体素，通过递归拆分，可以更精细地表示数据结构，这种分解方式使得八叉树在处理空间数据时十分高效。八叉树由于其具有充分的递推深度，可以以更高的精确度来表示目标的外形，因而被广泛地用于可视化。在离散单元格的计算中，每个单元的坐标都被保存为一个树节点，在三维的空间内，它可以被分割成 8 个子网格，且根节点的数据被其 8 个子节点都完整保存着。

本文将以 NC 格式文件数据建立八叉树模型。首先通过处理将所有数据提取而成，通常海洋与气象数据的 X, Y, Z 三个维度之间，层数最少的通常是 Z 轴，即海洋深度层数数据。

假设 X, Y, Z 三个维度的层数分别为 i, j, k 。将 i, j, k 三个数中最大的设为 n ，寻找 n 以上包括 n 的最小的 2 的倍数 x ，即 $n \leq 2^x$ ，设置一个边长为 2^x 的立方体，在八叉树数据结构中设置这个立方体为根节点，此时这个立方体能够完全容纳 NC 三维数据场组成的矩形。一般 MC 算法都是计算出三角面片来表示物体形状，在八叉树的建立过程中，当八叉树根节点中含有三角面片时，将八叉树分割成八个子节点，每个子节点均为长度为 $n/2$ 的小正方体，此时以子节点为根节点，将上述步骤重复。经过层层递归，在八叉树的叶子节点中，若存在三角面片，则为本文最后所需要的三角面片，在树的叶子节点中，每个立方体都含有三角面片，部分叶子节点所含的三角面片大于原始图形所求得的三角面片，相比之下，八叉树结构所计算出的面片数量将小于等于原始图像。从整体上说，一些平坦的琐碎三角面片被相对较大的面片取代，损失的精度可以忽略不计，且能够有效增加效率。

八叉树算法是一个递归算法，而递归算法的三个要素是写对算法的关键，三个要素分别为：确定递归函数的参数和返回值。确定终止条件。确定单层递归的逻辑。算法的流程图如图 3.11 所示。下文将八叉树结构结合递归算法的三个要素说明代码的编写思路。

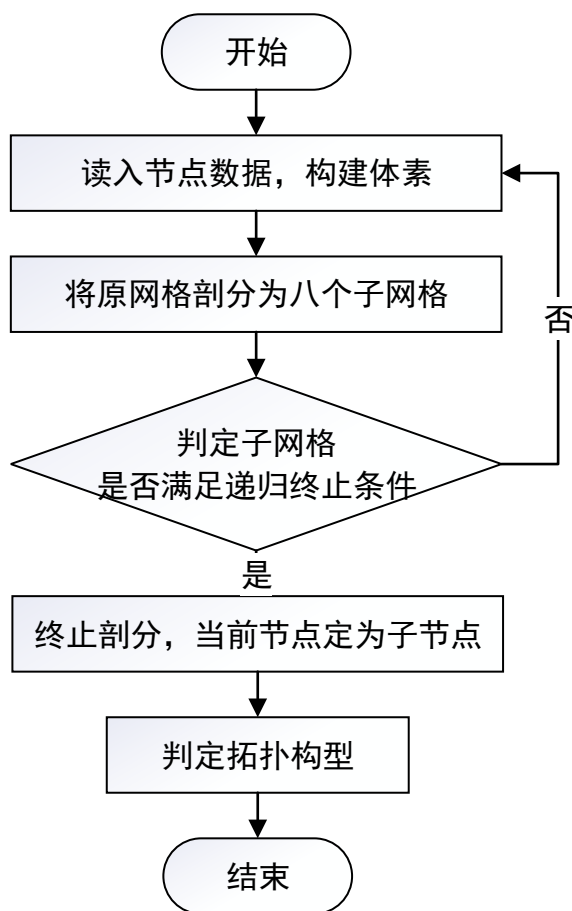


图 3.11 八叉树递归算法流程图

3.4.1 节点面片的判定

根据递归算法三要素分析, 参数毫无疑问是数据文件的参数类型, 返回值则是面片的坐标信息与法线信息, 里面具体细节不再赘述。由于子节点的计算与父节点是否为空相关, 不需要子节点的信息对父节点进行反推, 因此采用中序遍历的递归方法即可。而确定单层的递归逻辑是要确定相应的节点内是否存在三角面片, 当存在时, 需要计算面片位置信息及法线信息, 并储存在节点内, 这也是八叉树数据结构中每个节点所包含的信息。如果根节点的数据场中最大数据大于所求阈值, 且最小数据小于所求阈值, 那么该八叉树节点就含有三角面片, 反之, 如果最大数据小于所求阈值, 或最小数据大于所求阈值。

在进行八叉树递归时, 先将父结点分割成 8 个子结点, 然后保存相应的三角形面片信息。本文利用“解析-提取”的方法来判断节点中是否含有三角面片, 在某个节点所代表的三维数据场中, 获取数据场内的最大值与最小值, 当所求等值面的值介于最大值与

最小值之间时,节点非空,此时根据拓扑关系计算三角面片的位置信息与法线信息。反之节点为空,结束递归并将节点为空的信息返回至父节点中。

3.4.2 递归终止判定

在八叉树的构建中,面片信息均存储在各个节点内,且子节点是否存在与父节点是否为空有关,父节点的状态则与子节点无关。因此当某个节点达到终止条件时,立方体剖分停止,八叉树的创建过程也一并停止,当前节点为整棵八叉树的叶子节点。在所有递归分支终止后,收集所有叶子节点的信息,里面的面片信息为最终的网格信息,通过改进的拓扑结构可以对网格中提取出三角面片信息,当某个节点非空时,其一定存在某个节点非空,无需考虑子节点全空的情况。

如何设定递归的终止条件是至关重要的,常见的八叉树剖分终止条件分为三类:达到剖分深度、父节点内部变化缓慢、节点过于尖锐导致完全绘制所需面片数量过多。递归的终止条件极其重要,不同的终止条件会影响算法的效率,也会影响到八叉树的剖分精度以及生成的三角面片的精度。设定的具体条件需要结合不同的使用环境以及需求制定。

从海洋要素等值面的角度来分析,由于要素的数据变化形式繁多,对于一个要求具有广泛适应性的算法来说不可能只对某一类型的等值面形状进行分析,而应该提取等值面的几何特征来进行分析。考虑到海洋要素数据的等值面往往是平坦的,而出现“尖锐”等值面部分时证明数据发生急剧变化,需要将其绘制完全,因此在节点尖锐部分的终止条件不适合应用在本文中。因此在下面两种情况下,节点的递归计算将停止。

(1) 剖分深度达到需求

当剖分达到设定需求时是最基本的判定需求,若不对剖分的深度进行限制会导致剖分深度过高。由于使用场景的不同,所需绘制的数据大小会有较大差距,因此无法对递归深度进行设定,只能对递归停止条件进行设定从而保证剖分的自适应。当剖分的分辨率超过数据构成的最基本体素分辨率时,子节点无法找到对应的体素数据来进行拓扑判断,所以递归终止条件的第一个要求为判断剖分后的分辨率是否达到最小体素的数据分辨率,当节点达到时,停止剖分,当前节点成为叶子节点。

(2) 父节点面片变化平缓

在海洋要素数据中,有一部分等值面的绘制结果会非常平缓,当平缓的地形处被过度剖分时会造成面片数量大量增加但绘制的结果近乎相同的情况,出现该情况时应终止递归,避免内存浪费与效率延缓。但如何判断面片“平缓”是关键所在,理论上几乎所

有构型都有可能出现根节点的大面片接近其子节点变化的情况，但在绘制海洋数据等值面时，绝大部分出现的拓扑结构为构型 3，且数据较为平缓的区域中构型 3 出现的也最多。

当八叉树中某个节点被归类为构型 3 时，需要进行额外的判断，在递归前先一步进行剖分，若其所有子节点与该节点一样均为拓扑构型 3 时，被认为是“变化平缓”的部分，达到递归终止条件。除此之外，可以被判定平缓的剖分深度数值也需要进行人为控制。当剖分层数较少时，会出现节点计算深度不足的情况。如设置剖分层数为 1 即可进行判定平缓，此时会导致面片范围过大，虽然其子节点被剖分后都属于拓扑构型 3，但在子节点后继续剖分下会出现不同的情况，导致过于草率的将几个巨型三角面片取代大量的小型面片，这种提取结束递归的情况会使产生的等值面形状总体上绘制正确，但很多细小的特征结构被平整的面片所取代，绘制精度降低。但是在数值设定过大时，会导致计算过于深入，没有对效率进行过多的优化。在其他领域中，绝大部分领域的应用对深度数值的设定为 5 左右，但考虑到海洋领域中等值面反应出数据的变化情况，其内部细节至关重要，因此将深度数值设定为-2 层，即只有在最小的体素的父节点中才能够被判定“平缓”，在这样的设置下既能尽可能的保留等值面细节，又能一定程度上对算法效率进行提升。

3.5 仿真实验验证

基于本文的改进 MC 算法，首先要确定使用八叉树数据结构优化前后是否存在绘制错误，精度丢失等问题。如果上述二者均满足要求，在此基础上检验产生的三角面片数量区别和绘制速度的区别。使用八叉树数据结构改进后的算法对地形进行绘制，并使用等值线包围判断正确性，绘制结果如图 3.12 所示。

通过图 3.12 可知，八叉树数据结构改进后的 MC 算法在精度方面与原有 MC 算法几乎一致，观察平坦地形区域，并未发现明显精度损失。在正确性方面，从图中可以看出等值面被对应的等值线完美包围，也没有肉眼可见产生的空洞等问题，可以认为绘制的等值面结果是正确的。

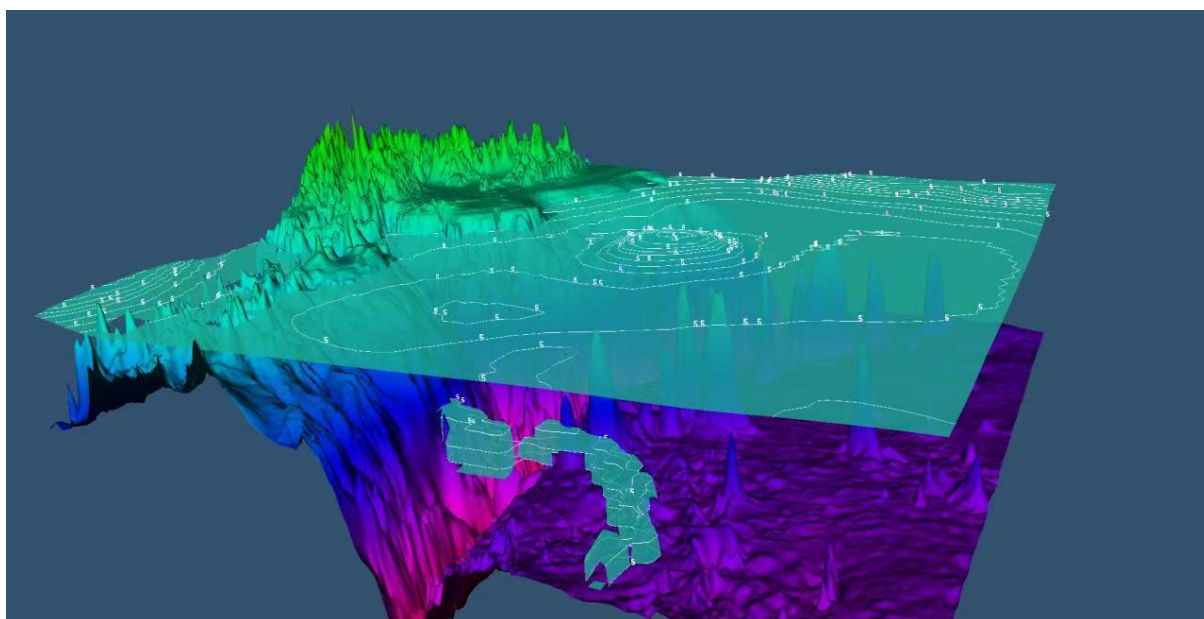


图 3.12 八叉树改进 MC 算法绘制地形图

3.5.1 改进 MC 算法的精确性验证

在精度与正确性得到保证的情况下，控制在同一区域，设置不同阈值利用两种方式进行绘图，对比三角面片的数量与正确性。

表 3.1 是在南海区域的海温数据下，调整设定的海温阈值，测定出传统 MC 算法与八叉树数据结构优化后的三角面片数量。

表 3.1 南海区域海温数据不同阈值下三角面片数量

海温阈值 (°C)	-5.5	-3.5	-1.5	0	1.5
传统 MC 算法 (个)	5634	7953	10324	8121	7633
八叉树结构优化 MC 算法 (个)	5601	7846	10182	8093	7621

对两种算法进行横向分析比较，温度设定最低值时，传统 MC 算法三角面片为 5634，在温度不断的升高过程中，面片数量不断增加，直到温度为-1.5 摄氏度时到达顶峰，此时数量为 10324 个，之后在温度继续升高时面片数量开始降低。这是因为海洋温度数据距离地表越远温度越低，当所求阈值较低时，得到的面片大多位于深度较高的位置，而过高的深度会使得接近海底区域，数据缺失值显著增加。当温度高于某个值后面片开始减少，这是因为开始靠近海面区域，海水接收到的光照开始增多，温度变化较快，符合该阈值的区域有所降低。

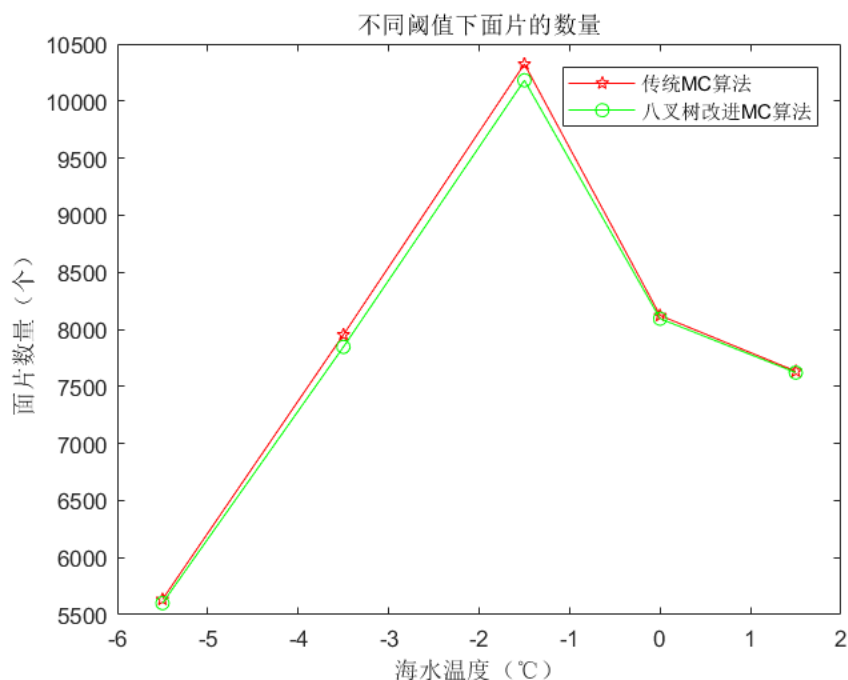


图 3.13 不同阈值下算法产生的三角面片数量

对两类算法进行纵向分析比较，得到不同阈值下算法产生的三角面片数量，如图 3.13 所示。在相同设定阈值下传统 MC 算法的面片数量略高于优化后的算法，综合面片数量损失约为 0.6%，这是在设定终止递归条件时考虑了可判定为“平坦”区域的条件。其中减少的面片数量为部分变化较慢的区域中部分琐碎的面片被较大的面片所替代，提前终止剖分，牺牲算法精度的情况下对算法效率进行提升。由于本文设定的深度较高，仅有部分最小体素单元被其父节点的立方体所替代，减少的面片数量少，从实际观感和面片数量来看损失的精度在可接受范围内。

3.5.2 改进 MC 算法的速度验证

为对比速度之间的差异，本文在 Windows 系统下均做速度测试，其 Windows 平台的配置如表 3.2 所示。

表 3.2 Windows 系统设备配置

机器名称	华硕飞行堡垒 FV63
系统	Win10 正版
CPU	英特尔酷睿 I7-7700HQ
内存	16G
内存频率	2400MZ
显卡	GTX1050
存储空间	1.5TB

本文在 Windows 平台下以表 3.2 的配置下,选择南海区域,不同阈值的海盐密度数据运行两类 MC 算法,并分别测试五次,测试的五次分别为不同区域范围的等值面。统计出传统 MC 算法与八叉树结构优化算法的后端运行所用时间,如表 3.3 所示。

表 3.3 Windows 平台下后端平台计算所用时间对比

测试次数 (次)	1	2	3	4	5
传统 MC 算法 (ms)	163	265	198	73	311
八叉树结构优化 MC 算法 (ms)	128	196	142	49	245

根据表 3.3 可以明显看出,本文所改进的 MC 算法不仅在精确性上有改进,速度上也有显著的效果。传统 MC 算法绘制时间平均在 202ms 左右,八叉树改进 MC 算法绘制时间平均在 152ms 左右,传统 MC 算法时间性能低于八叉树改进算法;对比传统 MC 算法,速度优化 24.8%,结合 3.5.1 小节看,八叉树改进算法略微牺牲判定“平坦”区域的精度,换来接近 25%的速度提升,说明八叉树结构用来优化传统 MC 算法的速率方面是可行的。为使对比结果更清晰,绘制 Windows 平台下两类算法速度柱状图,如图 3.14。

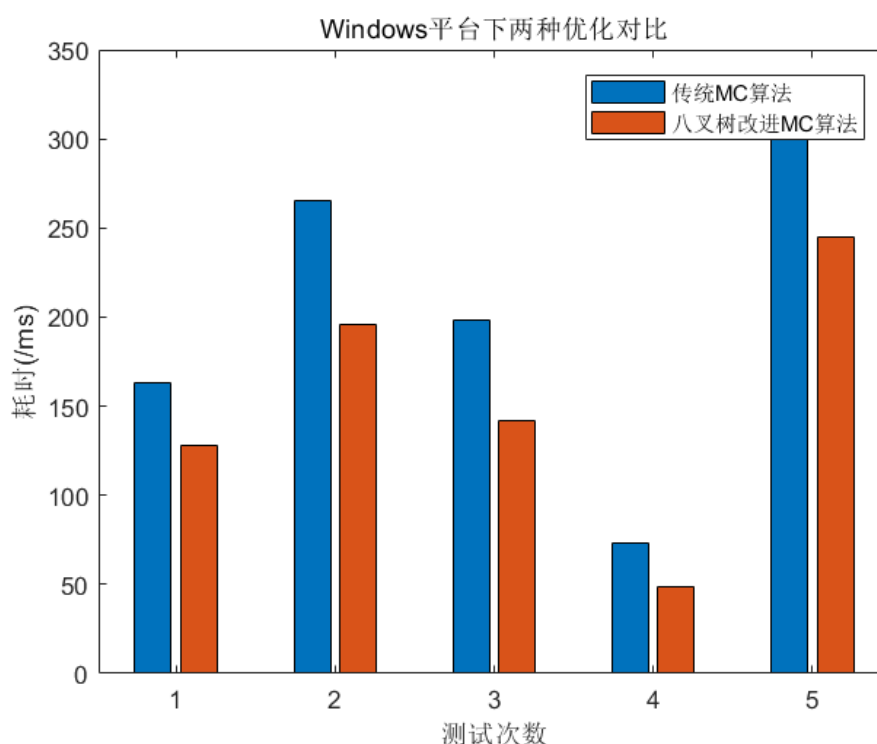


图 3.14 Windows 平台下两类算法速度

3.6 本章小结

本章首先对空间拓扑关系进行了说明,并对拓扑二义性的产生进行了分析,然后在

结合海洋要素的特点对 MC 算法拓扑二义性进行了改进,通过强制规定二义性面采取正连接的方式,对原有的 15 种拓扑结构进行细化分析,经过扩充后得到了 23 种拓扑结构,并对复杂鞍面的绘制证明了改进后算法的可用性。使用八叉树数据结构对算法效率进行改进,针对海洋要素的特点对剖分终止条件进行了设定,最后对改进后的算法进行仿真验证。

第4章 基于并行运算的效率优化

多线程是随着计算机硬件的发展的产物^[68]，由于海洋数据有着数据量庞大的特点，需要重点将 MC 算法的效率进行改进优化。第 3 章中先将 MC 算法的二义性问题进行改进，确保绘制图像正确后又使用八叉树数据结构进行效率提升，从绘制的效果和柱状图来看图像质量与原算法相比较为接近，并且算法的效率得到提升。但是八叉树数据结构是从算法本身出发进行优化，在合理应用计算机硬件进行效率优化方面仍然不足，多线程可以同时执行多个任务，充分利用多核处理器的性能，提高程序的整体执行效率。可以同时访问和操作共享资源，避免资源的浪费和冲突，提高资源的利用率。本章就根据传统 MC 算法与多线程的特点，结合第 3 章的拓扑改进与八叉树改进，采用合理的多线程优化方案。

4.1 多线程的基本概念

4.1.1 进程与线程

在早期的计算机中并没有多线程的概念，计算机只能单线接收操作指令，即使用者发送命令后，计算机进行单线处理，若使用者停止输入时，计算机原地等待。后来发展成使用者将一系列操作命令以清单的形式汇总，交予计算机处理，计算机按照命令的顺序进行按顺序执行，这个“清单”就叫做程序。

当用户同时将多个程序交给计算机进行处理时，早期的计算机按照程序交付的时间顺序执行，此时内存中只有一个程序，想要执行其他程序只有在前一个程序执行完成之后才能开始。为解决上述问题，人们提出进程的概念。进程其实就是一个个计算机执行中的程序，程序之间拥有相互独立的特点，因此进程与进程之间也是相互独立的。CPU 对进程之间的处理方式是给每个进程分配时间片段，当进程的时间片段使用完成时，该进程会被暂停，CPU 对其他进程给予时间片段，并在该片段内进行相应处理。由于时间片段的长度极短，通常都是毫秒级别。因此各进程之间的轮换十分快速，虽然每一个时刻只有一个进程在被 CPU 所执行，但在人类肉眼中看来进程之间在同时运行。

由于进程的出现，计算机的处理性能得到极大的提高，但进程的内部只能在同一时间执行同一件事，如果一个程序内部有多项任务需要执行，也只能按顺序逐个运行，于是提出线程的概念，线程是进程的内部划分，进程中包含一个或多个线程，每个线程能

够独立的执行子任务，线程也遵循时间片段的规则。相比于进程来说，线程拥有着轻量方便的特点，除此之外，线程之间数据共享，通信起来十分简单。进程与进程之间存在着数据隔离，并且进程间的通讯相对困难。

由于计算机硬件的发展，人们意识到 CPU 单核效率的提升越来越缓慢，若仅仅依靠对单核芯片的速度进行提升会造成一个 CPU 内核上集成的晶体管过多，导致 CPU 发热量过大，从而掣肘性能的进步，因此 CPU 开始往多核方向开始提升，具体内容如图 4.1 与图 4.2 所示。

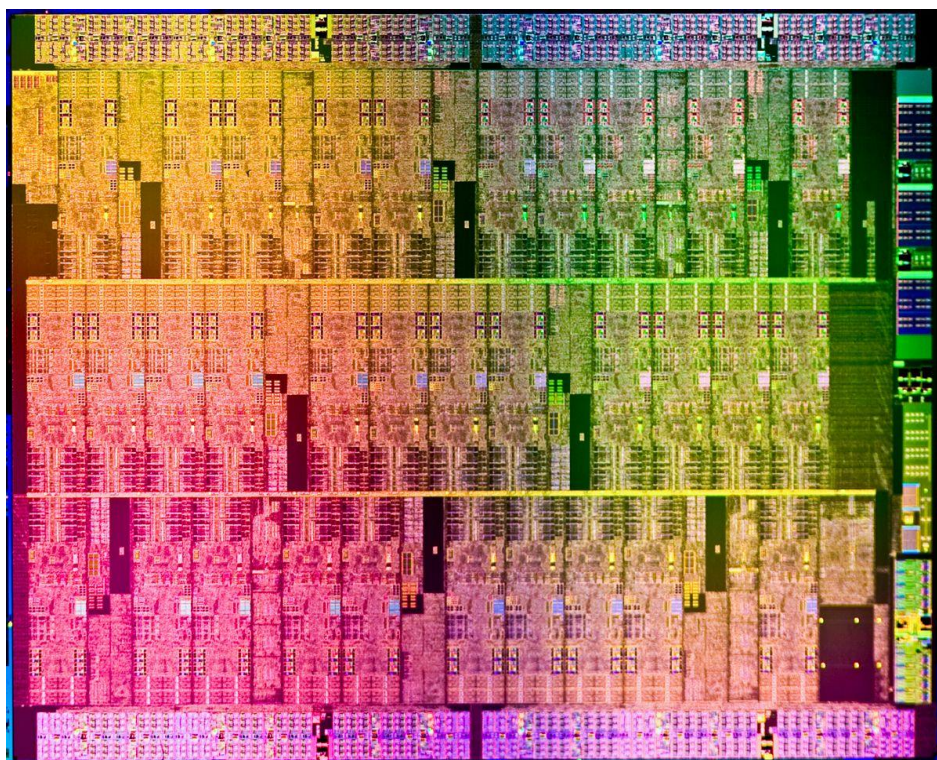


图 4.1 CPU 多核的发展

因为线程的出现，导致程序的运行可以同时处理多项任务，且 CPU 往多核方向进行发展，此时多线程在处理任务上存在并发和并行两种方式^[69]：并发是一个 CPU 同时执行多个任务，由于只有一个 CPU 工作，因此各个任务是在一个时间段内交替执行的，通过时间片段的方式轮流对多个任务进行处理，使得在人类视角内是同时处理的。线程的并发执行不需要多核 CPU 硬件支持。而并行是多个 CPU 同时执行多个任务。每个任务都被分配到不同的 CPU 核心之中并行执行，能够大量提高程序的运行效率。

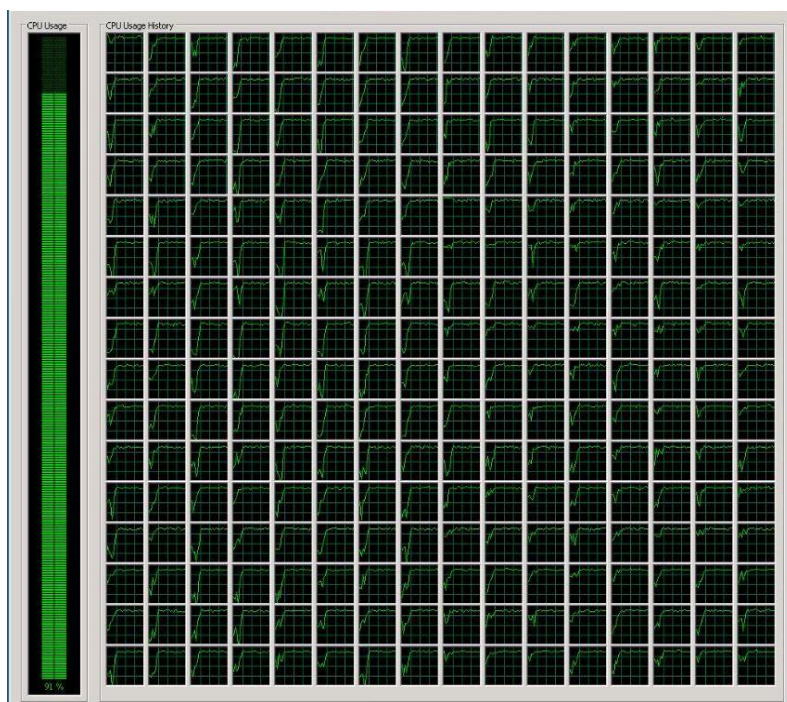


图 4.2 CPU 多核任务计算

为进一步提升 MC 算法的效率,本文采用多线程并行加速的方式,对算法进行优化,但多线程的并行运算需要将原本按顺序执行的一个任务合理拆分为多个任务进行处理,且由于线程之间的资源是共通的,多线程的执行很容易出现死锁,线程饥饿等现象,需要对每个线程,以及进程共享区等资源进行合理设计。

4.1.2 同步与异步

多线程存在并行与并发两种形式^[70],决定程序处理任务的方式。继续往程序内部进行细分,在程序运行期间,等待一个操作完成时,可以有同步和异步两种方式^[71],其中同步是指在多线程中,执行时间是有序的,当一个线程执行完成某个操作时,需要让步等待一部分其他线程完成任务,不能直接进行下一项操作的处理。而异步执行则不需要等待,直接转入执行其他程序的部分。同步的主要目标是协调在多线程环境下的工作,确保多线程条件下不会导致数据等共享资源不一致等错误。例如如果多个用户同时进行抢票操作时,如果没有对票数等共享资源进行做同步处理,可能会导致最后 1 张票被两人或多人同时购买。而异步的使用是为快速的执行后续操作,不需要做任何停留,提高执行效率。具体流程如图 4.3 所示。

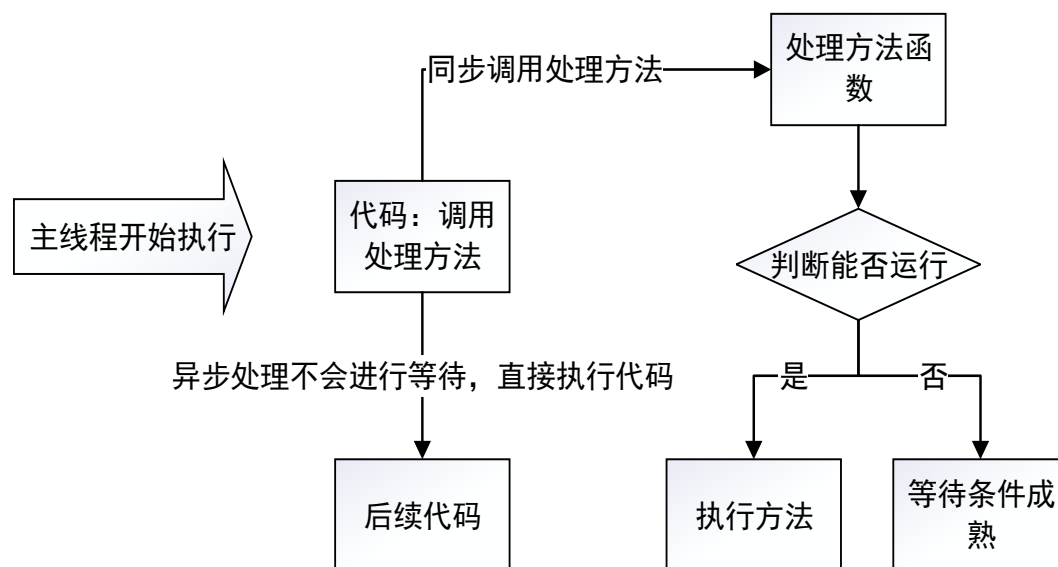


图 4.3 同步与异步流程示意图

为保证数据的一致性和完整性，需要合理的对同步进行运用，但过多的同步会使得线程进行长时间的等待，从而导致性能下降。因此平衡好异步与同步的代码是设计多线程程序的关键所在。其中常见的同步机制有互斥锁、信号量、条件变量、屏障等几种机制。

(1) 互斥锁：互斥锁能够允许多个线程使用同一份资源，但一次只能有一个线程访问这个资源，如果一个线程想要使用资源，它要先进行询问，若资源没有被使用，则该线程能够拿到资源，并对资源上互斥锁，当线程将资源使用完成后，将互斥锁释放；若资源已经被使用，则该线程因为资源被上锁而无法获取，从而进入等待状态。

(2) 信号量：信号量是一个能够表示系统中某种资源的变量，例如系统中有三台设备，则对设备的信号量设置为 3，当有线程请求使用设备时，若信号量数量大于 1，则允许线程使用设备，并将信号量大小减 1；若信号量为 0 时，则拒绝线程使用设备，此时线程进入挂起等待状态。信号量的使用必须通过原子命令实现，原子命令是无法被中断的特殊程序，通过原子命令能够保证信号量的数据安全性和完整性。

(3) 条件变量：条件变量是一种利用线程间共享的变量进行同步的一种机制，通常与互斥锁一起使用。由于互斥锁只有锁定和非锁定两种状态，条件变量可以设置一个逻辑条件，当条件满足时，线程会解锁且等待条件的变化，一旦另一个线程修改环境变量，就会通知其他被该条件所阻塞的线程。被唤醒的线程会重新上锁，测试条件是否满足。通过条件变量与互斥锁的配合使用，能够增加互斥锁表示的状态，降低资源的损耗。

4.2 多线程程序设计

由 4.1 节可知,多线程的设计需要合理平衡同步与异步之间的关系^[72],通过异步来提高处理效率,利用同步来保证资源正确性和完整性。除此之外,为保证线程安全,需要考虑以下几个方面:

(1) 原子操作:原子操作是一系列不可分割的操作,不会被其他线程中断。多线程环境下需要采用原子操作,避免操作被分割中断,导致数据错误等后果。

(2) 可见性:多线程同时访问一个数据时,需要确保该数据的变动是对所有线程来说都是可见的,防止线程间获取的数据资源是不一样的。

(3) 线程安全的数据结构:语言自带的数据结构类中,很大部分是为考虑简洁效率,放弃对线程安全的设计,例如 Java 中的 `HashMap` 类, `ArrayList` 类等等,而语言中往往会添加对应的线程安全的数据机构类,这些类别中具有和普通数据结构相同的功能,但由于考虑多线程之间的运行安全,效率会有所降低,与上述两类对应的线程安全结构为: `ConcurrentHashMap` 类与 `CopyOnWriteArrayList` 类。

多线程编程中需要高度关注线程安全的问题,尤其是在共享资源的情况下需要采用适当的线程安全机制,以保证数据的正确性和完整性。同时,需要注意线程安全机制的性能问题,避免过度使用线程安全机制导致性能下降。

4.2.1 优化流程分析

程序由单线程改写为多线程时,类比一个人做的工作改为多人同时完成,需要判定某一事件是否能够让多人同时完成,程序也类似。并非程序的所有步骤都能够由多线程实现,将改进后的 MC 算法流程进行分析,经过第 3 章内容改进后的 MC 算法流程图如图 4.4 所示。

根据流程图步骤进行考虑,读取数据部分由于 IO 口是串行通信,无法使用多线程进行改进;进入八叉树剖分计算面片步骤后,由于八叉树递归的特性,数据前后联系紧密,无法使用多线程改进;保存面片数据并发往前端也是基于 json 通信,与多线程无关;而前端绘制部分基于 Cesium 框架编写,底层语言为 JavaScript,不支持多线程,只能在一个线程中顺序进行。因此需要在八叉树计算面片流程之后,在将节点面片查找拓扑结构处采用多线程进行优化。

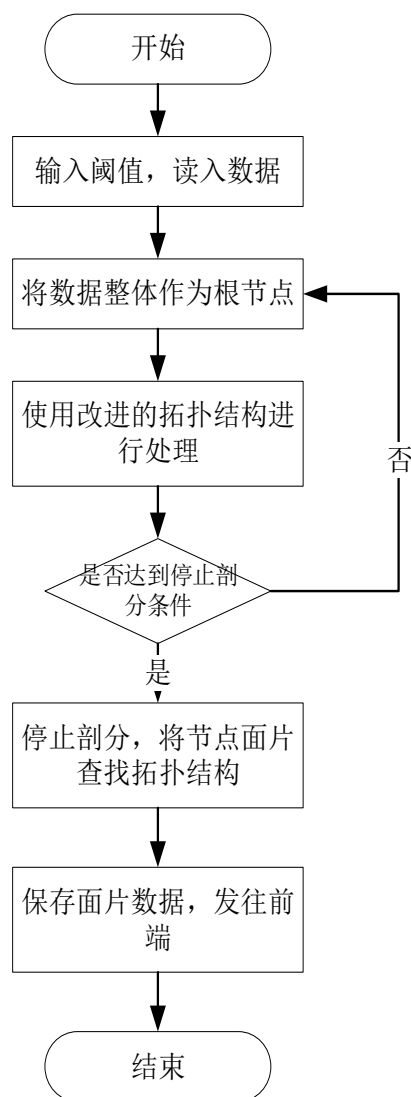


图 4.4 改进后的 MC 算法流程图

针对将节点面片处用多线程进行优化处理部分, 本文选择将所有拓扑构型分为四种类型处理。根据常见海洋等值面的形状分析, 将不同特点的构型归为一类, 生成一个专门的线程来针对性处理其负责的构型, 每个线程内部具有不同的线程处理函数。处理流程为: 判断子节点的数据进行判断构型分类, 分配给所针对的线程, 线程对相应构型进行面片计算, 流程图如图 4.5 所示。在四个线程同时运行处理下, 能过快速的将八叉树分割后的节点体素快速处理, 达到提高运行效率的目的。

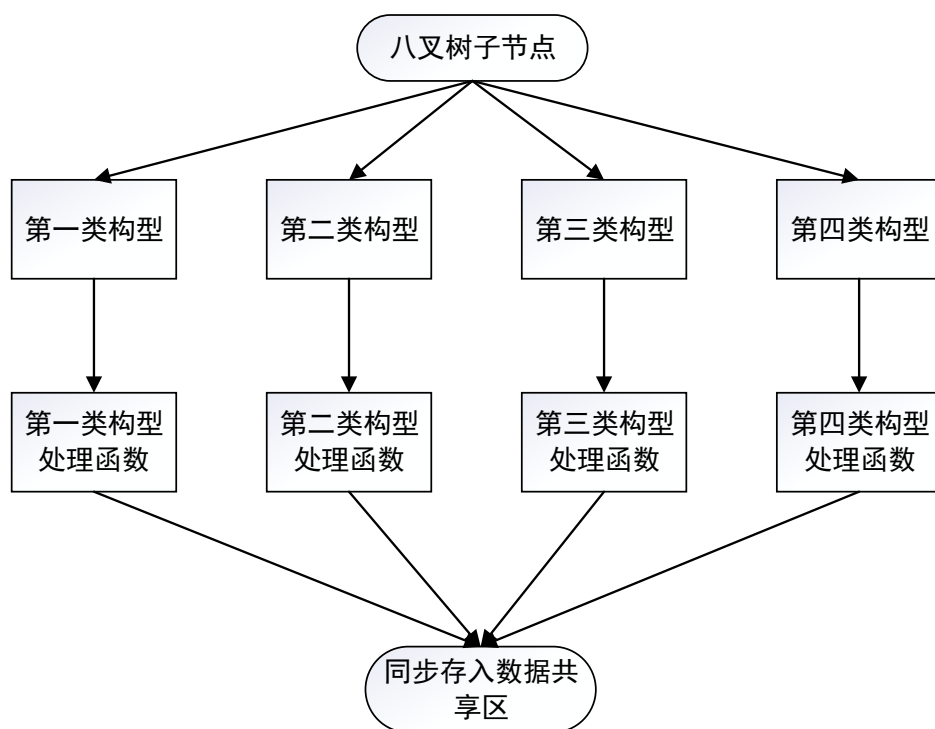


图 4.5 多线程处理流程图

4.2.2 拓扑构型的分类处理

在生成等值面过程中必须要处理体素，体素与等值面相交的位置等关键信息往往决定着处理的效率。在上一节可知，各个体素的处理是相对独立的，存在使用多线程进行优化的可能，因此将体素的不同拓扑情况进行归类分析处理。将 23 种构型分为四大类处理，第一类为构型 8；第二类为构型 2, 3, 2c, 3c；第三类为构型 4, 5, 9, 11, 14, 5c, 4c；第四类为构型 3, 6, 7, 10, 12, 13, 7c, 6c, 3c。其中特殊构型 0 与 0c 经过八叉树递归后并不存在，无需处理。

具体拓扑类型的划分是根据海洋数据等值面特点规则划分，第一类构型中只有构型 8，由于在大部分情况下，绝大部分海洋数据等值面的变化较为平缓，造成所有体素中构型 8 的数量最多，有时甚至占比高达 70% 以上。为提高多线程的计算效率，将情况最多的构型 8 以单独线程针对处理，如图 4.6 所示。第一类体素顶点为底面全为 0，顶面全为 1，索引值 $\text{Index} = 00001111 = 15$ ，棱边上的交点为 e_8, e_9, e_{10}, e_{11} ；其中蕴含两个三角面片，将对应棱边上的点 (e_8, e_9, e_{11}) ， (e_9, e_{10}, e_{11}) 连接形成三角面片后计算法线，将位置信息与法线信息存入共享区中。

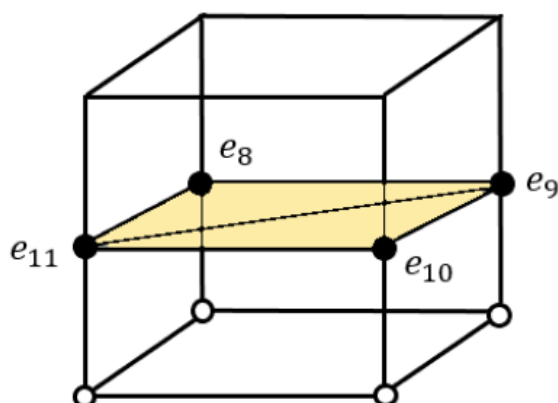


图 4.6 第 8 号构型示意图

第二类构型相比于第一类构型而言，在海洋环境等值面中出现的次数相对较少，但远比第三，四类多，且具有拓扑连接简单的特点，如构型 2，2c 只需将棱边中的三个交点连接起来就能在体素中形成三角面片。而构型 3 和 3c 能够从体素中得到两个面片，如图 4.7 以 3 号构型为例，该构型中点 2 和点 3 为 1，其余为 0；其索引值 $\text{Index} = 00001100 = 12$ 。其有四个与棱边的交点，但由于不存在二义性面，因此不需要进行二义性判断，且四个点是处于一个面内，和第一类情况类似，因此本文将顶点分为两组 (e_1, e_3, e_{11}) 和 (e_1, e_{10}, e_{11}) 存入共享区之中。

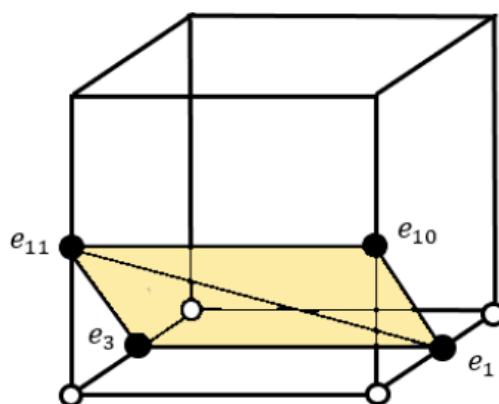


图 4.7 第 3 号构型示意图

第三类构型分类的体素单元中等值面面片情况复杂，往往是空间折叠多边形或多个离散三角面片。这类构型在海洋要素等值面中出现的情况很少，但处理相对复杂。空间折叠多边形以 5 号构型为例，其索引值为 $\text{Index} = 00000111 = 7$ ；通过分析可知该构型存在五个与棱边的交点，因此该构型中存在三个三角面片，此时因为交点个数众多，面片存在多种连接方式，但由于三个面片处在同一个面上，采用其中任意一种连接方式均可。离散三角面片以图 4.8 所示的 4 号构型为例，其索引值为 $\text{Index} = 00101000 = 40$ ；发现有六个棱边存在交点，但该构型中包含的面片并不共面， (e_2, e_3, e_{11}) 与 (e_5, e_6, e_{10})

为该构型中的两组面片。其余的构型也采用相同的方法进行处理，相比与前两类构型，第三类构型的面片数量较多，面对棱边上出现的众多点位，需要对三角面片进行划分，判断出正确的三角面片连接方式才能进行数据的存储工作。

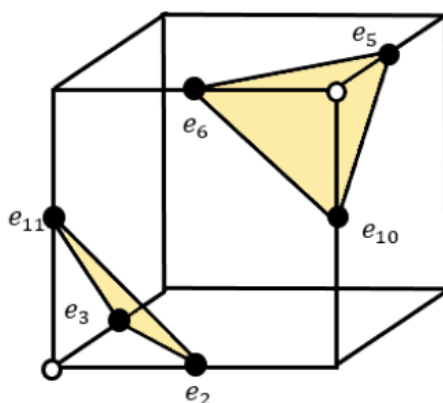


图 4.8 第 4 号构型示意图

第四类构型是所有情况中最为复杂的，但在等值面中出现的频率也是最少的，它们由于拥有二义性面，需要对二义性面进行判断。以图 4.9 所示 6 号构型为例，还是先进行索引值的计算， $\text{Index} = 00010101 = 21$ ；其棱边上共有 7 个顶点存在，根据共面性判断前，需要对面的二义性进行判定，由于第三章内容中本文以强制正连接的方式对二义性面进行处理，因此 e_1 与 e_2 两个点一定存在连接线路，故 (e_1, e_2, e_{10}) 分为一组，其余 (e_0, e_4, e_7) 和 (e_0, e_3, e_7) 组成两个共面的三角形面片。其余的构型也采用相同的方法进行处理，相比与第三类构型，这类构型不仅面片数量较多且存在二义性面，需要确保正确的拓扑连接方式。

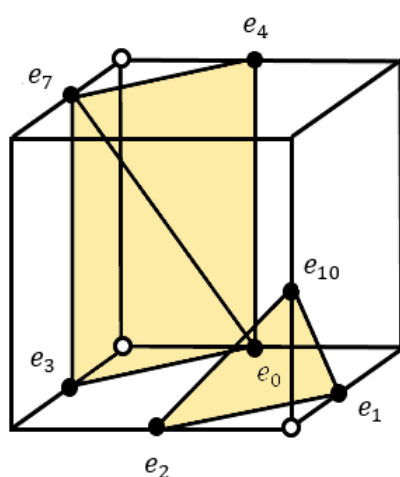


图 4.9 第 6 号构型示意图

4.3 多线程程序实现

4.3.1 数据共享区设计

多线程系统中的同步处理为保证资源的正确性与一致性的，需要重点分析。根据流程图分析，存在同步共享区设计的地方为保存数据处，单线程保存数据时不会出现错误，但多线程同时存储数据时，需要保证线程之间的存储顺序一致，否则会出现面片位置错乱，法线数据错误等问题。因此对该部分创建资源共享区。共享区代码流程图如图 4.10 所示。

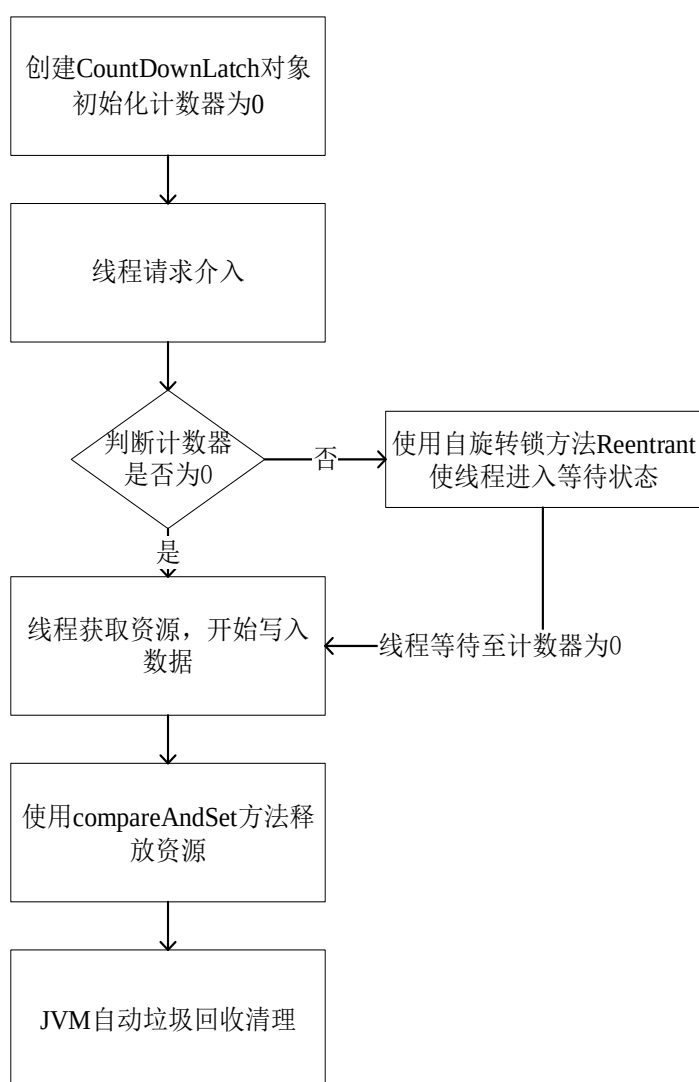


图 4.10 共享区代码流程图

首先，对原有流程中数据存储部分代码使用 CountDownLatch 工具类进行处理，通过该工具类内置的计数器来统计资源共享区是否存在线程进行使用。创建一个

CountDownLatch 对象，初始化计数器为 0，当某个类型的线程运行到需要往共享区中输入数据时，先检测计数器数字是否为 0，若为 0 时更新共享资源区的数据，并立刻返回。若不为 0 时证明共享区正在被其他线程传入数据，此时调用 `await()` 方法使得线程进入等待状态。由于每个线程传入数据的速率较快，且需要处理的体素往往较多，大量的进行线程状态切换会造成效率的降低，因此需要采用线程内部旋转锁来降低性能损耗。Java 中内置轻量级的旋转锁方法 `Reentrant`，当计数器不为 0 时，其他线程不再直接进入等待状态，而是通过保持线程自旋的形式来达到等待的目的。当之前占用的线程完成数据的写入时，使用 `compareAndSet` 方法将主内存中线程置空，相当于释放锁资源，允许其他线程进入。由于 JVM 中自带垃圾收集器，因此不需要手动清理线程中废弃的数据。

4.3.2 多线程并行速度验证与分析

在第三章中，对传统 MC 算法本身进行改进，保证正确性，精确性的同时获得速度上的提升。在本节中，将采用多线程方法并行处理的改进，希望进一步提升计算速率。为对比速度之间的差异，本文在 Windows 平台下做速度测试，其 Windows 平台的配置如表 3.2。基于 Windows 平台，使用南海区域不同阈值的海盐密度数据对后端平台计算速度进行测试，测试结果如表 4.1 所示。

表 4.1 Windows 平台下后端平台计算所用时间对比

测试次数（次）	1	2	3	4	5
传统 MC 算法（ms）	163	265	198	73	311
未使用多线程的八叉树优化 MC 算法（ms）	128	196	142	49	245
开启多线程的八叉树优化 MC 算法（ms）	87	182	94	31	170

由表 4.1 的测试结果可以看出，在速度上由于采用多线程并行方式进行加速，所以速度上也有显著的效果。通过对比可以未使用多线程的 MC 算法和开启多线程的 MC 算法所用时间，明显看出多线程并行运算后的算法效率比单线程要高，但第二次出现差距不明显的情况，很大概率发生线程饥饿现象，即产生的大部分面片都集中在某个线程之中，其他线程一直在等待，此时运算速率与单线程接近。

由表 4.1 中同时可以看出，开启多线程效率有显著提升，但提升幅度不一致。当发生线程饥饿现象时，速度提升幅度在 8% 左右，当线程任务分配比较均匀时，速度提升幅度能够达到 27% 以上。开启多线程与八叉树结构优化的算法对比最传统 MC 算法来说，速度提升能达到 40% 以上。综上所述，本文提出的开启多线程并行处理来提升算法效率的方法是有效的。

4.4 跨平台验证运行效率

由第三章内容可知, 本文将传统 MC 算法本身进行改进, 保证正确性, 精确性的同时获得速度上的提升, 在 4.3 节中采用的多线程并行处理也对速度有进一步的提升, 本节将验证算法的可移植性。

改进 MC 的算法本身便具有的跨平台特性, 但由于本文使用 Java 语言对该算法进行实现, 通过 Java 语言中内置的 JVM 虚拟机的跨平台特点, 轻松实现了算法的可移植性与跨平台性。为验证本文改进的算法的跨平台可移植性, 本文需要在 Windows 与 Linux 平台下做出测试。

由于 Windows 平台的配置与相应的实验数据如表 3.2。Linux 平台的相关配置见表 4.2, 测试的实验数据见表 4.3。

表 4.2 Linux 平台设备配置

机器名称	Ubuntu 虚拟机
系统	CentOS 7 64 位
CPU	2
内存	2G
内存频率	2400Mz
存储空间	20G

表 4.3 Linux 平台下后端平台计算所用时间对比

测试次数 (次)	1	2	3	4	5
未使用多线程的八叉树优化 MC 算法 (ms)	226	342	244	83	420
开启多线程的八叉树优化 MC 算法 (ms)	175	330	168	64	348

在 Linux 平台下, 由于虚拟机配置调度问题, 速率比 Windows 系统稍慢, 但由于 java 语言跨平台的特点, 系统完全能够使用。根据表 4.3 中可以看出, 在 Linux 系统中, 多线程仍然起到优化速度的作用。

4.5 本章小结

本章首先介绍了多线程的基本概念, 然后对多线程程序进行了设计。通过对剖分完成后的所有叶子节点进行多线程处理, 根据海洋要素的特点将所有拓扑构型分为四类, 每一类开启单独的线程进行针对性处理。设计数据共享区来处理多线程对共享资源的冲突。最后分别在 Windows 系统和 Linux 系统对改进后的算法进行仿真验证。

第5章 基于改进 MC 算法的海洋可视化系统设计与实现

本章基于改进的 MC 算法设计并开发一套海洋要素可视化系统,对海洋要素等值面进行绘制。系统分为数据载入模块,三维体视化绘制,图像后续处理三大模块。通过前后端分离的形式完成,前端基于 Cesium 框架设计而成。后端基于 SpringBoot 框架搭建,并进行封装打包,拥有跨平台的特性。

5.1 海洋可视化平台研究现状

目前,国际上有关科研单位和企业已经在海上模拟和可视化系统和平台方面进行较多的研究,并获得一些优秀的结果^[73]。当前,美国宇航局的 World Wind、国内的 EV-Globe 和 iOcean 等都是较为成熟和为人所熟悉的可视化软件(平台),它们都有自己的特点,可以为海洋数据的可视化应用提供支撑。

(1) World Wind

World Wind 是由美国宇航局(NASA)推出的一项开源、自由、可用于自主研发的软件开发套件(SDK)。该系统支持开发者在三维空间中的快捷、便捷的构建三维地球,并可对其进行交互展示,使其具有良好的可扩充性,易于实施,从而为各种空间数据的表达、操作、可视化和分析等提供强有力的工具。

World Wind 为现实生活中的数据提供可视化,操纵和分析的方式。同时,该系统还提供空间数据的各种表示方式(如散点图、数据分布图等);该系统可以将所选择的时段的资料以动态的方式呈现出来,将资料的变化状况以一种非常直观的方式呈现出来,并且还可以对世界各地的台风、飓风变化、海洋气温的变化进行仿真。本系统所提供的资料具备较强的自我更新功能,可对气象、灾情等进行即时监控,为防灾减灾工作提供参考。

(2) EV-Globe

EV-Globe 是由国遥公司自行开发的一款大规模的三维空间信息系统,在数据浏览、矢量数据查询、三维分析等方面具有突出的优点,在国内同类平台中居前列。其发布的 EV-Globe Ocean 版本能够实现对海洋表面光线和水流的仿真,获得逼真的视觉感受。可以在水下进行 3D 绘制。该系统具有较强的综合能力,可完成天、地、海三个维度的综合管理和综合表示。

(3) iOcean

中国“数字海洋”公共平台(iOcean)是国内第一个面向“数字海洋”的公共业务平台。该项目以全球球面为基础,采用现代的立体表示方法,构建“数字海床”、“数字水体”、“海洋军事”、“虚拟海洋馆”等功能模块,将海洋中的各种要素(如海底、水体、表层等)进行“可视化”呈现,并以可视化的方式呈现许多海洋现象。相比于外国,iOcean是一种让公众更好地理解 and 认知海洋的方式,也是一种普及海洋知识和提高公众海洋意识的重要手段,可以更好地服务于国内公众。

5.2 系统整体框架设计

5.2.1 系统需求分析

通过对比上述三个主流平台,可以发现一个基本的海洋三维体视化系统应该包含:数据读取模块,等值面提取模块,图像处理模块三大部分。数据读取模块的功能为:在预置文件夹中放入需要进行等值面绘制的各种数据文件,将所有数据文件以日期和要素类型命名。当前端传来数据读取请求时,对预置文件夹中的相关数据文件进行检索,若寻找到目标文件则把海洋三维规则数据场中的数据依序读入至内存中,用于后续等值面的提取。等值面提取模块的功能为:将内存中的数据按照使用者需求进行等值面提取,快速提取出等值面中所有面片点的位置数据与相关的法线数据,其中提取等值面的算法为本文提出的改进 MC 算法。图像处理模块的功能为:通过对等值面提取模块中传入的所有三角面片位置数据进行绘制,并根据面片的法线数据对等值面进行渲染,将图像展示给使用者。

5.2.2 系统框架设计

根据上一小节对系统进行设计,海洋环境数据大多以 NetCDF 文件格式进行存储,该文件格式以.nc 为后缀。因此本文基于 Java 语言编写数据读取模块,通过载入 Java 中常用的 NCCDF.jar 依赖包中的方法实现数据读取功能;等值面提取模块,通过 Java 语言对本文中改进的 MC 算法进行实现,并使用 SpringBoot 框架对该模块进行后台封装,用于后续与前端之间的通信;图像处理模块,基于 JavaScript 语言进行编写,使用 Cesium 框架搭建前端,对等值面数据进行绘制渲染,并呈现在三维球体之上,主界面应简洁大方且易于操作理解。整体框架流程如图 5.1 所示。

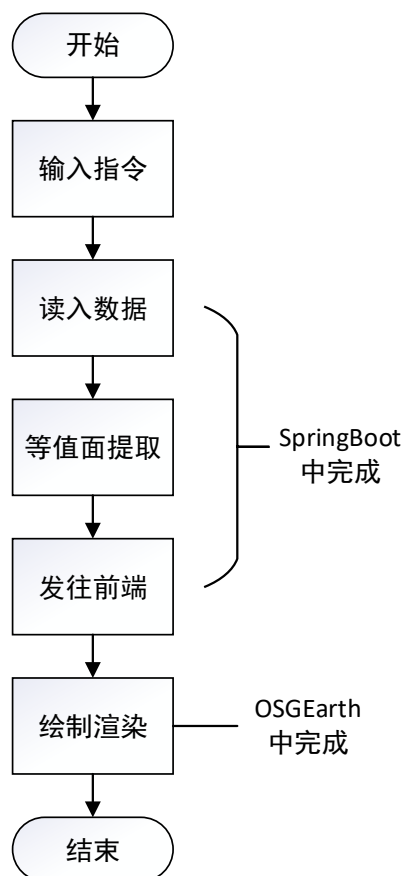


图 5.1 系统整体流程

5.3 基于改进 MC 算法的海洋可视化系统

系统的开发环境为 Windows10 系统，后端基于 Java 语言编写，开发工具为 IDEA，前端基于 JavaScript 语言编写，开发工具 VSCode，系统使用的数据来源为全球海洋数据。数据格式为.nc 文件，每个文件中包含时间、空间位置、海洋环境要素值等头文件信息。

5.3.1 海洋数据读取

海洋数据大多以 NetCDF 文件格式进行存储，该文件格式以.nc 为后缀，最初设计用于存储气象领域，由美国大学大气研究协会开发，在后续的发展下已经成为很多数据采集软件使用的文件格式。NetCDF 文件通常会将数据的各个维度作为标签独立存储，例如海洋温度要素中会将包含某个区域海洋温度的经度，纬度，高度（或深度），时间等标签。通过识别这些标签，各种算法语言中有着很多方法能够快速提取一系列数据用于后续等值面绘制，例如基于 Java 语言的 NCCDF.jar 依赖库等。

当用户在前端按钮中设定需要读取数据的信息，前端将信息转化为标准空间 WKT 坐标格式，并备注数据的名称与类型，以 POST 请求的形式传给后端。后端接收到 POST 请求时，读取 POST 请求中的信息，并解析 WKT 格式的数据，得到用户需要载入的数据的名称和数据范围。由于后端基于 SpringBoot 框架设计而成，因此只需将后端 Maven 仓库中添加 NCCDF.jar 依赖库，之后便可调用 NetcdfFile.open()方法来读取数据，通过将解析获得的数据类型信息作为参数传入 openNC.findVariable()方法中，将数据范围作为参数传入 value.open()方法中，即可成功载入数据。由于每种 NC 数据的数据量不同，字符类型也不同。为方便后续对等值面的提取，本文将数据全部转化为三维的 double 类型数据。具体流程如图 5.2 所示。



图 5.2 数据读取流程

5.3.2 基于 SpringBoot 的后端系统搭建

SpringBoot 是一个用于简化和加速 Java 应用程序开发的开源框架，它提供许多便捷的功能和工具，使得后端搭建变得更加高效和灵活^[74]。SpringBoot 是基于 Spring 框架的

扩展,旨在简化 Spring 应用程序的配置和开发。它通过自动配置和约定大于配置的原则,提供快速构建可独立运行的应用程序所需的一切。在进行后端搭建之前,需要准备好项目环境。

首先设置 Java 开发环境、安装和配置 SpringBoot 框架,并创建一个新的 SpringBoot 项目。本文将介绍标准的项目结构,包括主应用程序类、配置文件和资源文件的位置。主应用程序类为 IsoMain 类;其余应用程序类包括存储数据内容的 IsoSurfaceData 类;用于数据内容转化的 IsoSurfaceDataDTO 类;包含所有本文中改进 MC 算法单元的 IsoSurfaceUnit 类;以及用来进行单元测试的 Demo 类;配置文件为 Pom.xml,其中包含 NCCDF.jar 等依赖库;资源文件为 resource 文件夹内部的所有文件,其中多数为前端页面文件,如图 5.3 所示。

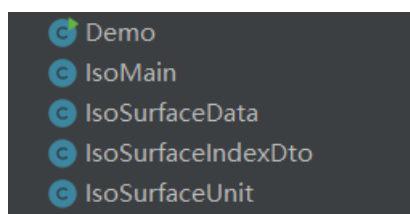


图 5.3 整体后端类

在完成后端搭建后,需要对应用程序进行测试和调试,Demo.java 类将完成该项工作,该类将使用 SpringBoot 提供的测试工具来进行单元测试和集成测试,并可以使用 Lombok 调试器来进行调试应用程序,以解决潜在的问题和错误。在通过单元测试后,对所有依赖库,代码等内容进行打包处理,得到 Jar 类型文件,方便进行跨系统部署到生产环境中。

5.3.3 基于 Cesium 的前端页面搭建

本文主要内容为传统 MC 算法进行海洋要素的针对性改进,在可视化平台里位于等值面提取模块,因此在前端模块中只需搭配合适的框架进行绘制渲染即可。

前端显示模块基于 JavaScript 语言编写,通过 Cesium 框架实现前端页面搭建。Cesium 是一个开源的地理信息系统 (GIS) 工具包,提供强大的地图渲染和可视化功能。它支持各种数据格式,并提供全面的工具集来显示和分析地理空间数据。具体步骤分为 Cesium 的安装和配置、HTML 页面的创建、必要依赖的引入以及 JavaScript 代码的实现来初始化和配置地图。在设置 Cesium 所需的安装和配置过程方面,要下载包括必要的软件依赖项、获取 Cesium 软件包的说明以及配置环境的指南,并完成配置。接着进行创建前端 HTML 页面,包括 HTML 文件的基本结构、包含必要的 CSS 和 JavaScript 库

以及创建用于容纳地图的容器元素。为使用 Cesium 的功能，需要在 HTML 页面中引入一些依赖项，通过内容分发网络或引用本地文件的方式来包含所需的 JavaScript 和 CSS 库。

在基于 Cesium 搭建前端页面的过程中，需要对 HTML 页面的编写，通过 HTML 语言对页面进行设计，其中包含了等值面绘制区域，以及加载海洋环境数据区域以及查看体数据环境区域。之后需要针对 HTML 语言预置的按钮编写 JavaScript 代码，每一个 JavaScript 代码实现按钮所需要的功能。完成 JavaScript 代码的编写后，前后端的联系通过 HTTPS 协议中的 POST 请求实现。搭建完成后的前端页面如图 5.4 所示。

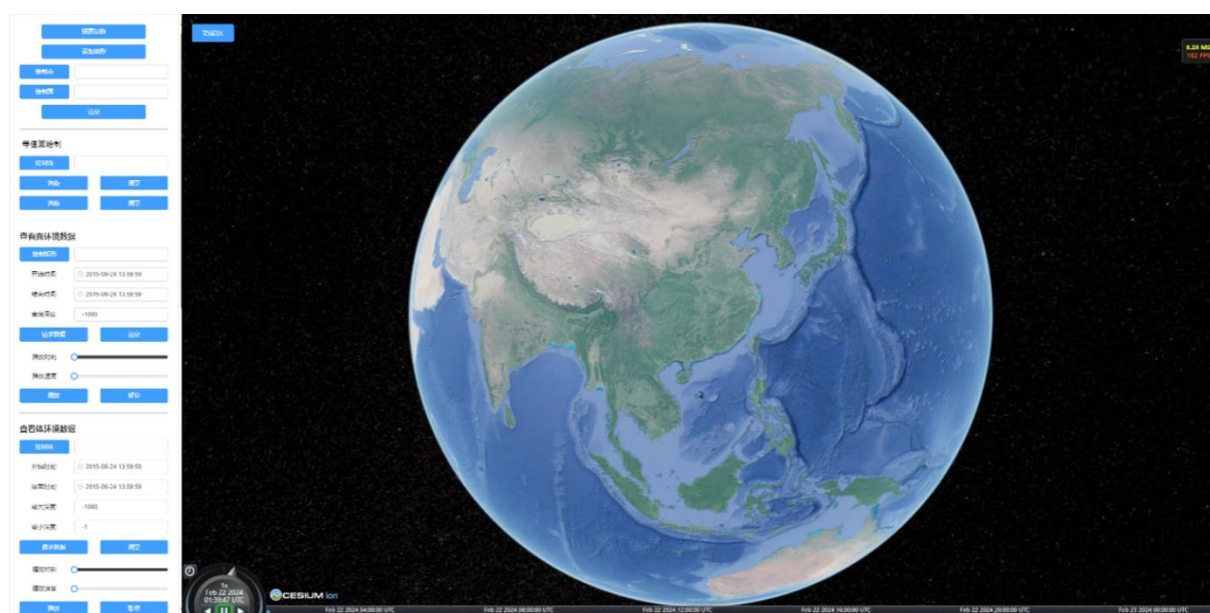


图 5.4 前端搭建页面展示

在实现前端页面之后，由于对海洋要素均位于水面以下区域，将等值面图案按照固有海拔高度绘制会导致图片均处于球体内部，不便进行观察，因此对所有面片进行等值拔高操作，提升的高度大小为最后一层海洋数据的深度。

将数据值和颜色值之间建立映射函数，通过对所求阈值的极限进行颜色映射，其中所有阈值中最小阈值设定为 0，最大阈值设定为 1，中间阈值使用线性插值方法获得其具体值，并对应映射函数找到属于自己的颜色。本文选取如图 5.5 所示的红绿蓝色标图。



图 5.5 红绿蓝色标图

最后对东经 110 度至东经 130 度，北纬 10 度至北纬 30 度的南海海域数据进行测试，通过设定 20 个等间距跨越的温度值为所求阈值，使用改进后的 MC 算法进行等值面提取，并绘制渲染，结果如图 5.6 所示。其中靠近海平面处的等值面温度阈值较高，对比色标图后呈红色和橙色；靠近海底处的等值面温度阈值较低，对比色标图后呈深蓝色。靠近海南岛区域水深不足，提前抵达海底，导致数据出现缺失，而绘制出的等值面左上角处提前出现缺失，右下角为远离海南岛区域，能够获得较多海温数据，能够绘制出深处的等值面图像。

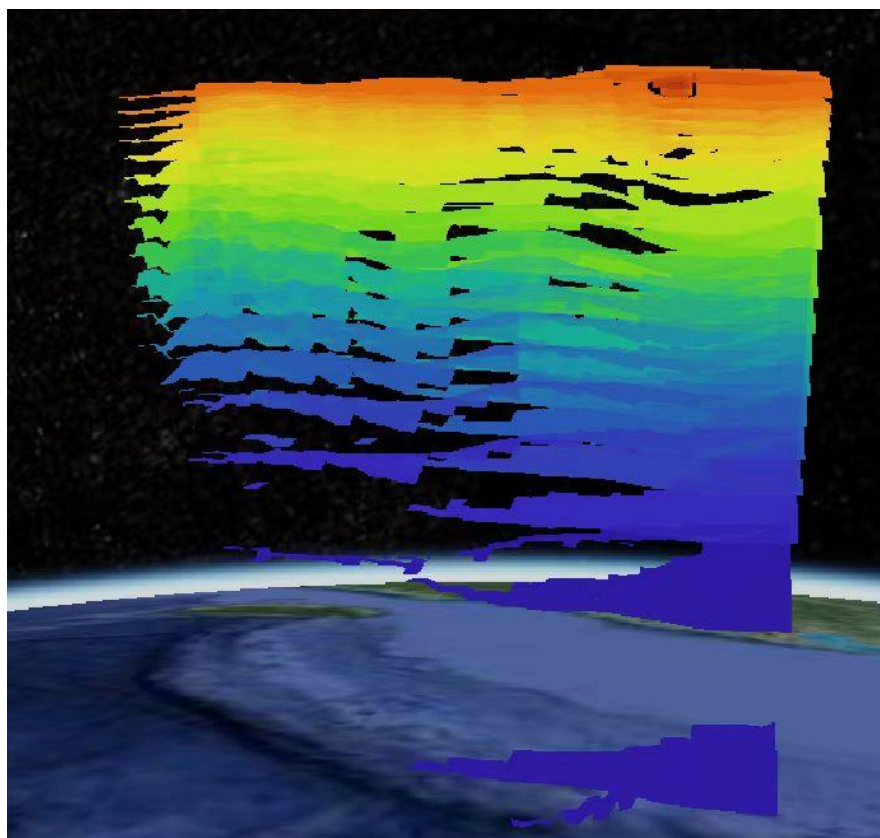


图 5.6 南海区域海温等值面

5.3.4 可视化系统测试

为了进一步证明该可视化系统的实用性，对可视化系统进行速度测试与稳定性测试：基于 Windows 平台，使用东经 110 度至东经 130 度，北纬 10 度至北纬 30 度的南海区域不同阈值的海盐密度数据对可视化系统计算速度进行测试，其中平台配置如表 3.2 所示，速度测试结果如表 5.1 所示。

表 5.1 Windows 平台下可视化系统计算所用时间

测试次数（次）	1	2	3	4	5
可视化系统所用时间（ms）	651	1343	712	322	1309
后端系统所用时间（ms）	87	182	94	31	170

通过表格数据可知，可视化系统所用总时长取决于后端系统计算所产生的面片数量，相比后端运算，前端对图像进行渲染对时间的占用较高。保持上述测试所产生的图形，持续运行可视化平台，24 小时后系统仍能正常运行，通过上述测试很好的证明了可视化平台的实用性与稳定性。

5.4 本章小结

本章基于文中改进的 MC 算法，对海洋要素三维可视化系统软件进行了设计和开发。首先对现有的主流海洋可视化平台进行分析并对比它们各自的优缺点。然后分析了本文所设计框架的需求，将系统分为数据载入，等值面提取，图像绘制三大模块。系统基于 Java 语言，以 SpringBoot 为框架开发后端系统；基于 JavaScript 语言以 Cesium 框架开发前端页面。对多层等值面进行绘制，验证了软件的可用性。

结 论

随着技术的发展,海洋数据量的增加,人们有快速观察海洋数据变化的需要。本文先是对现有海洋可视化方法的特点进行详细的分析,选择其中最适合绘制等值面的 MC 算法。之后对 MC 算法的研究现状进行研究,分析算法的不足并针对海洋可视化的特点进行针对性的改进。在解决拓扑二义性存在错误的缺陷上,本文在原本的 15 种基本构型上对部分存在二义性面的构型进行细化分析,新增 8 种拓扑构型,解决二义性带来的空洞等问题;为增加算法效率,本文使用八叉树数据结构来避免大量无效体素的遍历,并采用多线程的方式,针对不同类型的拓扑构型使用多线程处理;针对海洋数据高度跨越不均的问题提出插值方法,并最后对海洋三维可视化系统的前后端进行搭建,并封装成 jar 文件用于跨平台部署,本文主要的工作及结论如下:

(1) 针对传统 MC 算法二义性存在错误的缺陷,本文对二义性产生的底层原因进行分析,通过对传统 MC 算法中的 15 种基本拓扑构型里存在二义性面的构型进行细化分析,并强制对所有二义性面采取正链接的连接方式,新增 8 种拓扑构型,解决由于三维物体二义性而产生的空洞等错误的现象。在对绘制复杂度高,容易出现拓扑连接错误的地方进行针对性绘制,并使用拥有面二义性的等值线进行包围,最终结果显示改进后的算法能够有效保证拓扑连接的正确性。

(2) 针对传统 MC 算法需要对所有基本体素进行遍历,计算过程冗杂的特点,使用八叉树数据结构优化算法效率,改进后的算法通过对所有的数据进行自上而下的剖分,避免大量空白体素的计算,并提出提前终止递归的条件,大大提高算法效率。在图像精度方面:改进后的算法面片数量减少约 0.6%,损失的精度在可接受范围之内;在运行效率方面:改进后的算法效率提升约为 24.8%,在运行效率方面有着显著提升。

(3) 面对计算机硬件领域的发展,本文采用多线程并行计算方式对算法进行优化,根据拓扑构型的自身特点以及海洋要素的特点进行分析,将所有构型分为四个类型,对每个类型采用不同的线程进行针对处理,对效率的提升取决于海洋数据等值面的面片分布与所用 CPU 的多核性能等,通常在 8%到 27%之间。多线程与八叉树结合的算法对效率提升在 40%以上。

(4) 基于文中改进的 MC 算法,对海洋要素三维可视化系统软件进行了设计和开发。系统基于 Java 语言,以 SpringBoot 为框架开发后端系统;基于 JavaScript 语言以 Cesium 框架开发前端页面。对多层等值面进行绘制,验证了软件的可用性。

因为个人精力和时间有限,本文存在一些不足之处,需要进一步改善,例如:本文新增的 8 种拓扑构型是通过对二义性面进行强制性规定得到,并没有从图像本身出发,如果对算法效率要求降低可以对数据进行深入分析,从图像本身出发得到二义性面的连接方式。多线程优化对效率的提升不够稳定,优秀时能够提升 30%,低迷时仅仅提升 8%;因为 GPU 平行计算的特性与八叉树数据结构自顶而下的递归产生冲突,没有完成使用 GPU 优化效率的功能。

参考文献

- [1] 孟祥镇, 马鑫, 吴凯通, 等. 基于 Web 的医学影像三维可视化系统的设计与实现[J]. 电脑知识与技术, 2024, 20(04): 59-62.
- [2] 陈永华, 李思忍. 区域性海洋要素持续测量平台新技术概述[J]. 海洋科学, 2009, 33(04): 76-80.
- [3] 徐盼盼, 梅元勋, 赵庆庆. 基于 GIS 的海洋灾害预报预警系统应用研究[J]. 水利信息化, 2023, (06): 39-44.
- [4] 高峰, 付忠良. 基于改进移动立方体的医学图像三维重建算法[J]. 计算机应用, 2013, 33(S1): 201-203+213.
- [5] 徐效文, 王永华. 基于等值面优化重建医学图像的改进算法[J]. 微计算机信息, 2009, 25(25): 4-6.
- [6] 戴荣. 医学图像三维重建算法研究及实现[D]. 昆明理工大学, 2009.
- [7] 刘敏. 基于 VTK 的医学图像三维重建的研究[D]. 燕山大学, 2013.
- [8] Mittal R, Malik V, Singla G, et al. 3D reconstruction of brain tumors from 2D MRI scans: An improved marching cube algorithm[J]. Biomedical Signal Processing and Control, 2024, 91105901-.
- [9] Shuqiang L, Dawei Z. The Representation of Dynamic Factor Marine Element Structure Upgrading Model Based on Two Kinds of DSGE Models[J]. Journal of Coastal Research, 2020, 112(sp1): 337-339.
- [10] 李志炜, 傅军, 韩洪祥. 海洋环境数据三维可视化系统设计与实现[J]. 海洋测绘, 2019, 39(06): 68-72.
- [11] 谢先伟, 毛红. 海洋环境监测数据自动分析系统的设计与实现[J]. 舰船科学技术, 2016, 38(12): 121-123.
- [12] 韩成虎, 韩成龙, 王梦. MC 算法在 CT 医学图像三维重建中的应用[J]. 现代计算机, 2013(13): 20-22.
- [13] Lorensen W E, Cline H E. Marching Cubes: A High Resolution 3D Surface Construction Algorithm[C]. Conference on Computer Graphics and Interactive Techniques. ACM, 1987: 163-169.
- [14] Ilinka H, T. M M, Mirjam C, et al. Prevalence and Content Analysis of Guidelines on

- Handling Requests for Euthanasia or Assisted Suicide in Dutch Nursing Homes[J]. Archives of Internal Medicine, 2000, 160(3): 317-322.
- [15] 张宇, 马丽红, 余英林. 三维模型空间占有体的描述及显示预筛选算法[J]. 华南理工大学学报(自然科学版), 1997, (07): 103-107.
- [16] 左小清. 地下采空区三维模型的建立与可视化[D]. 昆明理工大学, 2001.
- [17] 张冬生, 杨耀权, 何晓燕, 等. 煤场体积测量中三维模型的建立[J]. 计算机测量与控制, 2003, (03): 168-170.
- [18] 刘岩, 马洪滨, 李金生, 等. 基于工程柱体的三维地质建模方法[J]. 金属矿山, 2009, (01): 101-103.
- [19] 吴炳方, 张明金, 李新功. 地理信息系统的发展[J]. 地理学报, 1994, (S1): 633-640.
- [20] 席学强, 王润生. 用于遥感图像人造目标识别的三维建模方法研究[J]. 计算机学报, 2001, (06): 638-644.
- [21] 陈克虎. 融合 RGB-D 视频中多模态特征的人体行为识别[D]. 南京邮电大学, 2015.
- [22] 尚福华, 杨彦彬, 杜睿山. 基于 TIN-Octree 的三维地质模型构建方法研究[J]. 计算技术与自动化, 2019, 38(04): 121-125.
- [23] 张桂林, 陈大克, 冯佐海, 等. Arc View 中栅格图像与矢量图的融合技术[J]. 桂林工学院学报, 2004, (02): 195-197.
- [24] 成毅. 基于 DEM 的三维地形建模及集输管道路径优化研究[D]. 东北石油大学, 2019.
- [25] 李景轩, 周登极, 肖旺, 等. 燃气轮机机理-数据混合建模方法研究[J]. 热能动力工程, 2019, 34(12): 33-39.
- [26] 程朋根, 龚健雅, 史文中, 等. 基于似三棱柱体的地质体三维建模与应用研究[J]. 武汉大学学报(信息科学版), 2004, (07): 602-607.
- [27] 程朋根, 龚健雅. 地勘工程 3 维空间数据模型及其数据结构设计[J]. 测绘学报, 2001, (01): 74-81.
- [28] 侯恩科, 吴立新. 面向地质建模的三维体元拓扑数据模型研究[J]. 武汉大学学报(信息科学版), 2002, (05): 467-472.
- [29] 吴立新, 史文中, Christopher Gold. 3D GIS 与 3D GMS 中的空间构模技术[J]. 地理与地理信息科学, 2003, (01): 5-11.
- [30] M J. Additional reference to marching cubes [J]. Computer Graphics, 1988, 22(2): 72-73.
- [31] Cignoni P, Ganovelli F, Montani, et al. Reconstruction of topologically correct and adaptive trilinear surfaces[J]. Computers and Graphics, 2000, 24(3): 399-418.

- [32] Natarajan B K. On generating topologically consistent isosurfaces from uniform samples[J]. The Visual Computer, 1994, 11(1): 52-62.
- [33] Van Gelder Allen, Wilhelms Jet al. Topological considerations in isosurface generation[J]. ACM Transactions on Graphics, 1994, 13(4): 337-375.
- [34] hernyaev E V. Marching cubes33: construction of topologically correct isosurface[R]. Geneva: European Organization for Nuclear Research, 1995.
- [35] Matveyev S V. Resolving the topological ambiguity in approximating the isosurface of scalar function[C]. Proceedings of Visualization and Machine Vision, Seattle, 1994: 18-21.
- [36] Nielson G M. On marching cubes[J]. IEEE Transactions on Visualization and Computer Graphics, 2003, 9(3): 283-297.
- [37] 梁秀霞, 张彩明. 拓扑结构正确的三线性插值曲面的三角片逼近[J]. 计算机研究与发展, 2006, 43(3): 528-535.
- [38] Lopes A. Accuracy in scientific visualization[D]. Leeds: University of Leeds, 1999.
- [39] Lopes A Brodlie K. Improving the robustness and accuracy of the marching cubes algorithm for isosurfacing[J]. IEEE Transactions on Visualization and Computer Graphics, 2003, 9(1): 16-29.
- [40] ilhelms J, Van Gelder A. Octree for faster isosurface generation[J]. ACM Transactions on Graphics, 1992, 11(3): 201-227.
- [41] Schroeder W J, Zarge J A, Lorensen W E, et al. Decimation of triangle meshes[J]. Computer Graphics, 1992, 26(2): 65-70.
- [42] R Shu, Zhou Chen, Kankanhalli M S. Adaptive marching cubes[J]. The Visual Computer, 1995, 11(4): 202-217.
- [43] Montani C, Scateni R, Scopigno R. Discretized marching cubes[C]. Proceedings of the Visualization, Washington, DC, 1994: 281-287.
- [44] 薛强, 蔡文立, 石教英. Marching Boxes: 一个多精度等值面抽取算法[J]. 计算机辅助设计与图形学学报, 1998, 10 (1): 7-14.
- [45] ontani C, Scateni R, Scopigno R. Decreasing isosurface complexity via discrete fitting[J]. Computer Aided Geometric Design, 2000, 17(3): 207-232.
- [46] Pascucci V, Bajaj C L. Time critical isosurface refinement and smoothing[C]. Proceedings of the 2000 IEEE symposium on Volume visualization, Salt Lake City, 2000: 33-42

- [47] Rodlie Ken, Wood Jason. Recent advances in volume visualization[J]. Computer Graphics, 2001, 20(2): 125-148.
- [48] 杨文. 基于体绘制的三维医学图像可视化技术研究[D]. 华中科技大学, 2007.
- [49] Jian Jun Xiang, Fei Deng, Tian Xin Ning. The Research of the Hybrid Volume Rendering Algorithm Based on GPU Technology[J]. Applied Mechanics and Materials, 2014, 3334(610):722-728.
- [50] 孙薇薇. 光线投射体绘制算法关键技术研究[D]. 天津理工大学, 2006.
- [51] Harvey R H P. Raycasting architectures for volume visualization[J]. IEEE Trans on Visualization and Computer Graphics, 1999, 5(3): 210-223.
- [52] Levoy M. Efficient ray tracing of volume data[J]. ACM Trans on Graphics, 1990, 9(3): 245-261.
- [53] 黄展鹏, 周苏娟, 赵洁. 直接体绘制加速算法综述[J]. 计算机与数字工程, 2008(03): 28-30, 73, 164.
- [54] 晏靖靖. 三维地震数据场直接体绘制的方法研究[D]. 南京理工大学, 2004.
- [55] 刘侠, 刘欢, 王淼淼等. 改进 Marching Cubes 算法的椎骨 CT 图像三维重建与可视化[J]. 哈尔滨理工大学学报, 2019, 24(06): 84-92.
- [56] 赵玲, 石雪, 夏惠芬. 基于 Marching Cubes 算法的数字岩心建模方法研究[J]. 石油机械, 2018, 46(10): 97-102.
- [57] 毕硕本, 陆源, 曾晓文, 等. Marching Cubes 改进算法及其气象三维模拟[J]. 系统仿真学报, 2017, 29(07): 1405-1410+1418.
- [58] 热孜万古丽·夏米西丁. Marching Cubes 算法研究[J]. 科技视界, 2012, (34): 66+108.
- [59] 孙伟, 张彩明, 杨兴强. Marching Cubes 算法研究现状[J]. 计算机辅助设计与图形学学报, 2007, (07): 947-952.
- [60] 翟巍, 马行, 穆春阳, 等. 基于八叉树编码的铸件点云融合简化方法[J]. 组合机床与自动化加工技术, 2023, (10): 6-10.
- [61] 林义忠, 谢震鹏. 基于 YOLOv7-mask 和 ORB-SLAM2 的语义八叉树地图构建[J]. 机床与液压, 2023, 51(23): 86-91.
- [62] 杨茂男, 贾庆轩, 李旭龙, 等. 基于上下文强化八叉树网络三维模型语义分割算法[J]. 计算机应用研究, 2021, 38(12): 3586-3589+3596.
- [63] 王金鑫, 秦子龙, 曹泽宁, 等. 基于八叉树的修正克里金空间插值算法[J]. 郑州大学学报(工学版), 2021, 42(06): 21-27.

- [64] 张满囤, 燕明晓, 马英石, 等. 基于八叉树结构的三维体素模型检索[J]. 计算机学报, 2021, 44(02): 334-346.
- [65] 杨晓文, 李静, 韩燮, 等. 基于八叉树的卷积神经网络三维模型分割[J]. 计算机工程与设计, 2020, 41(09): 2663-2669.
- [66] Kim S, Saurabh K, Khanwale A M, et al. Direct numerical simulation of electrokinetic transport phenomena in fluids: Variational multi-scale stabilization and octree-based mesh refinement[J]. Journal of Computational Physics, 2024, 500112747-.
- [67] Cheng-Hau Y, Kumar S, Guglielmo S, et al. Optimal surrogate boundary selection and scalability studies for the shifted boundary method on octree meshes[J]. Computer Methods in Applied Mechanics and Engineering, 2024, 419116686-.
- [68] Wang X, Wang X, Yu M, et al. MesoGRU: deep learning framework for mesoscale eddy trajectory prediction[J]. IEEE Geoscience and Remote Sensing Letters, 2021, 19: 1-5.
- [69] 张迎平, 高国贤, 陆一峰等. 基于区间树硬件加速索引的 Marching Cubes 算法[J]. 计算机辅助设计与图形学学报, 2012, 24(07): 871-878.
- [70] Su C, Peng C, Lei D, et al. A New Semi-Analytical MC Model for Oceanic LIDAR Inelastic Signals[J]. Remote Sensing, 2023, 15(3): 684-684.
- [71] Asyabi E, Sharafzadeh E, SanaeeKohroudi S, et al. CTS: An operating system CPU scheduler to mitigate tail latency for latency-sensitive multi-threaded applications[J]. Journal of Parallel and Distributed Computing, 2019, 133232-243.
- [72] Catalán S, Herrero R J, Igual D F, et al. Fine-grain task-parallel algorithms for matrix factorizations and inversion on many-threaded CPUs[J]. Concurrency and Computation: Practice and Experience, 2022, 35(27):1-10.
- [73] 郭沫含, 王维, 杨丰侨. 我国海洋数据平台综述研究[J]. 企业科技与发展, 2023, (10): 9-12+30.
- [74] 游晶, 邱淑丽, 李敬文. 基于 SpringBoot 的校园智慧报修系统的设计与实现[J]. 电脑知识与技术, 2023, 19(32): 50-52+65.